arXiv:2611.11509v1 [cs.RO] 8 Nov 2026

REVIEW THE EPISODES, REFINE THE LESSONS: SIMPLE MEMORY EVOLUTION FOR EMBODIED AGENTS

Yifei Yao[1*] **Yude Zou**[1,2*] **Xing Gao**[1†] **Guanbo Jiao**[1,3] **Yukai Wang**[1]
Jianming Ma[1,2] **Weinan Zhang**[1,2] **Chunhua Shen**[1,4]

[1]Shanghai Artificial Intelligence Laboratory [2]Shanghai Jiao Tong University
[3]Fudan University [4]Zhejiang University
[*]Equal contribution [†]Corresponding author

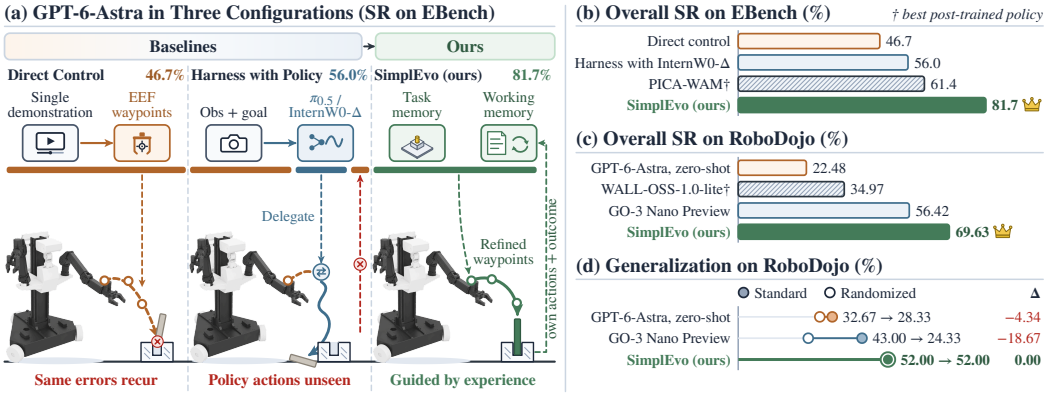

Figure 1: GPT-6-Astra in two baseline configurations, direct control and a harness with policy, and with SimplEvo, whose working memory is written on training episodes and then fixed. (a) Each configuration with its EBench success rate (SR) and InternW0-Δ as the harness policy. The bar under each shows which component commands the robot, and the MLLM does not see the actions of a delegated policy. (b) Overall SR on EBench and (c) on RoboDojo, where † marks the strongest post-trained policy and zero-shot denotes the official entry of GPT-6-Astra. (d) SR on the 12 RoboDojo Generalization tasks with standard and randomized configurations.

ABSTRACT

Recent studies of embodied agents based on frontier multimodal large language models (MLLMs) develop harness engineering around the MLLM, such as harnesses with post-trained policies and recursive self-improvement loops. Yet whether these additions exceed the native capability of the MLLM remains unexplored. We introduce SimplEvo to explore how far an MLLM can improve under direct control when it retains its execution experience in self-written text memories. It requires no weight updates, post-trained policies, or privileged information. The MLLM writes a coordinate-free *task memory* once from a single demonstration. Under a strict train–test protocol, it then creates a bounded *working memory* on training episodes only and refines it after each episode with lessons on procedure, completion, and recovery. Both memories stay fixed at test time. On EBench, SimplEvo lifts the success rate (SR) of GPT-6-Astra under direct control from 46.7% to 81.7%, and tops the RoboDojo leaderboard with 69.63% SR and a score of 75.32. Compared with memories that also store past visual frames, SimplEvo reaches a similar SR with text memory alone while using fewer tokens. With its memories fixed, SimplEvo also generalizes to unseen layouts and randomized scene configurations. Moreover, the working memory transfers across models: written by GPT-6-Astra, it can be reused as is by weaker MLLMs and improve their performance without any adaptation of their own.

1 INTRODUCTION

Generalist robotic manipulation is a central problem of embodied intelligence. Vision-language-action models (VLAs) [2, 3, 25] and world action models (WAMs) [55, 61, 63] align their manipulation skills through post-training on benchmark-specific trajectories [5, 30, 38]. In contrast, recent studies show that frontier multimodal large language models (MLLMs) directly operate robots to complete tasks without parameter updates. They command the end effector from camera views and language instructions [21, 24, 31, 32, 40, 66], explore and adapt their behaviors to the observed results [9, 18], and recover from errors within an episode [12, 60]. These findings have motivated a growing range of embodied agents built on frontier MLLMs.

These agents fall into two lines of work. The first is *code-as-policies*, which began with language models writing robot code [29] and composes perception modules and motion primitives into executable programs. In systems such as Code as Policies [29], COAP [22], and RAX on RoboDojo [23], the synthesized programs run at test time without the MLLM in the loop, so they cover only anticipated scene variations and failure modes. Recent works such as CaP-X [12] and PyRUA-Lean [47] therefore execute code online with the MLLM in the loop. Even so, code-as-policies systems typically rely on privileged information beyond RGB views and proprioception, such as metric depth, camera parameters, and object sizes, or on estimates from calibrated depth sensors and vision foundation models [26, 43, 58, 59], to which the resulting programs are sensitive. Frontier MLLMs, by contrast, already perceive and reason about space from RGB views and proprioception, which raises the question of whether routing perception through intermediate models is necessary at all.

The second is *harness engineering*, which extends the harness around the MLLM with post-trained policies or external improvement loops. Some systems build *harnesses with policy* that expose post-trained policies as callable tools [53, 67], so the MLLM plans while the policy executes low-level actions. Each delegated call returns only its final observation and status, so the actions of the policy never enter the context of the MLLM, and the harness can constrain the capability of the underlying MLLM. Other systems adapt self-evolving language agents into embodied *recursive self-improvement* (RSI) frameworks that revise context, tools, or control logic through revision loops around the agent [8, 33, 37, 49]. These frameworks iterate on external harnesses and rarely measure the base model under direct control first. Some self-evolving designs adapt on the test scenes themselves [54]. Because the evaluator then returns feedback on the scenes used for evaluation, test-scene information can leak into the evolved system, and its gains no longer show whether the acquired experience generalizes to scenes the agent has never attempted. It thus remains unclear how far a frontier MLLM can go under direct control on its own, and hence how much of this added machinery is necessary.

Both lines of work focus on either exploiting the coding ability of the MLLM or wrapping it with post-trained policies and improvement loops, thereby overlooking the rising capability of the MLLM itself to act directly as a general-purpose robot controller. We refer to this setting as *direct control*, where the MLLM commands the end effector directly using RGB views and proprioception, without external perception modules or a post-trained policy. Direct control returns the result of every executed command to the MLLM, so the closed-loop feedback and within-episode recovery are explicit. After a missed grasp or a failed insertion, the MLLM compares the commanded end-effector coordinates with the resulting camera views, diagnoses the error, and adjusts its next command [60]. These adjustments and the experience of each trial, however, vanish when the episode ends, so each episode starts anew. Can a frontier MLLM unlock strong manipulation under direct control simply by inheriting and evolving the in-episode adjustments and trial experience across episodes, relying only on its native multimodal capabilities?

To show that recording and revising its own memory is enough for a frontier MLLM to excel under direct control, we present SimplEvo, illustrated in Figure 2. SimplEvo keeps the native direct-control loop unchanged and introduces only two compact text memories that the MLLM writes itself during adaptation on the training set. From one training demonstration, the MLLM formulates a coordinate-free *task memory* once, and after each training episode, it reflects on its own trajectory in one turn to rewrite a bounded *working memory* of transferable lessons. A strict train–test protocol confines adaptation and its outcome feedback to training scenes, none of which appears in the test set. Both memories are then fixed, and every test episode reads them without reflection turns or memory

updates. Throughout adaptation and testing, the agent observes only RGB views and proprioception and requires no post-trained policies, weight updates, or privileged information.

With its working memory fixed after adaptation, SimplEvo outperforms post-trained policies and other MLLM agents on the test episodes of EBench [15] and the test layouts of RoboDojo [6]. On EBench, SimplEvo raises GPT-6-Astra from 46.7% under direct control to a success rate (SR) of 81.7% across its 510 test scenes while reducing tokens per success, against up to 56.0% SR for harnesses with policy and 61.4% SR for the strongest post-trained policy. On RoboDojo, after adapting on 15 training layouts per task, SimplEvo leads the public leaderboard across 50 test layouts per task with 69.63% SR and a score of 75.32.

This paper makes three contributions:

- We propose SimplEvo, a two-stage memory framework for direct control, in which the MLLM derives a task memory without coordinates from a single training demonstration and rewrites a bounded working memory through its own reflection on every training episode, without post-trained policies or weight updates.
- Under a strict evaluation protocol with train–test separation, native step limits, and no privileged information, SimplEvo ranks first on both the EBench and the RoboDojo leaderboards, ahead of post-trained VLAs, WAMs, and other MLLM agents, while using fewer tokens per success than direct control.
- We show that SimplEvo performs well with simple text-based memories and, with these memories fixed, generalizes to unseen layouts and randomized scene configurations. Moreover, the working memory created by a frontier MLLM can be reused by weaker MLLMs to improve their performance.

2 Related Work

Harnesses and self-improvement for embodied agents. Beyond the code-as-policies systems and harnesses with policy discussed in Section 1, harnesses also coordinate post-trained policies through memory [57], and direct control can command end-effector targets [24, 32, 66] or semantic action units [7]. Self-improving embodied systems differ in what they change. They revise tools, harnesses, or code policies during development and score the fixed result [22, 37, 49], co-evolve guidance with harness modules [10, 28, 48, 51, 52], or distill trajectories into policy weights [4, 42, 56], and Zhou et al. [71] classify such systems by which component a retained change updates and how broadly its gain holds. Controlled audits find that harness evolution does not consistently outperform test-time scaling at equal budget, that frequent updates can fail to improve held-out performance [54, 69], and that selection on small, noisy evaluations admits spurious gains [16, 45]. SimplEvo keeps tools, harnesses, code, and model weights fixed. It acts under direct control and updates only the text working memory that the MLLM reads across episodes.

Frontier MLLMs as robot policies. Frontier MLLMs now control robots without being trained on robot data. For example, GPT-6-Astra [40] commands end effectors directly on RoboDojo [6, 66] and learns from demonstration blocks in context [31], while other MLLM agents write robot commands from saved procedures [24] or refine a revisable model of the robot body [32]. Yao et al. [60] profile such an agent on EBench [15] across direct control and harnesses with policy, and they find that its failures concentrate in physical execution. This study measures how far an agent under direct control improves once its execution experience persists across episodes.

Memory and experience evolution. Agent memory systems differ in how they represent and update experience across episodes. Language agents improve without weight updates by accumulating verbal reflections [46], extracting insights from successful and failed trajectories [68], or curating evolving playbooks [65]. In multimodal interactive environments, VISTA [20] preserves a lossless visual history of past observations, so carrying it across manipulation episodes lengthens the context with every executed command. Embodied agents store experience as executable skills [50], validated program repairs [33], policy repositories of prompts, tools, and control code [11], per-task programs [53], or a knowledge package that GPT-6-Astra writes once from training demonstrations and reads unchanged in every test episode [44]. Unlike reflections, insights, and playbooks distilled from language-level actions, or knowledge written from demonstrations, the working memory of

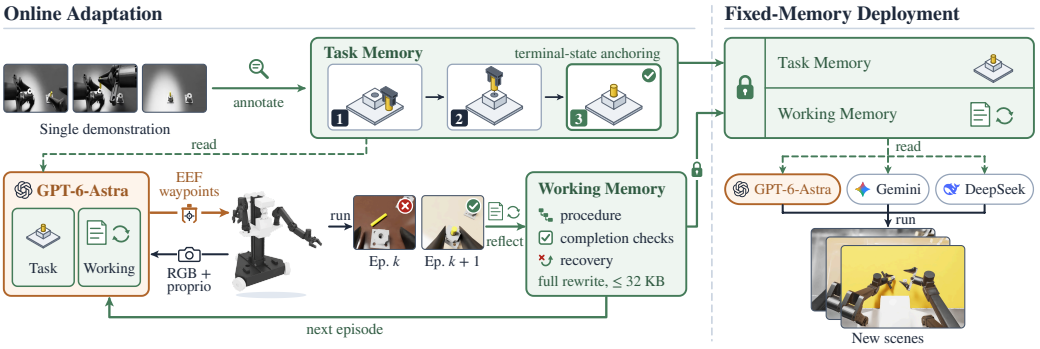

Figure 2: Overview of SimplEvo. During online adaptation (left), an annotator instance of GPT-6-Astra reviews one training demonstration and writes a coordinate-free task memory of the stages and the completed state, and after every training episode, GPT-6-Astra rewrites the working memory from the trajectory and outcome of that episode into lessons on procedure, completion, and recovery. Once fixed (right), both memories are read in new scenes, and other MLLMs read the working memory of GPT-6-Astra without modification. All panels show peg insertion on EBench.

SimplEvo distills the low-level commands that the MLLM itself issued, is rewritten in full within a fixed size after each adaptation episode, and stays fixed at test time.

3 Method

As shown in Figure 2, SimplEvo equips direct control with two memories written by the MLLM, each overcoming a limitation of isolated episodes. The *task memory* supplies the order of stages and the final state of a task before the first attempt, and the *working memory* preserves the corrections that would otherwise vanish when an episode ends.

3.1 Direct Control under a Strict Information Protocol

Agent under direct control. The agent pairs a fixed-weight MLLM core F with an agent harness that specifies its observation and action space. Under *direct control*, the MLLM issues every robot command itself instead of calling a post-trained policy. At decision step t of the k-th episode on task τ, the MLLM receives the observation $o_t = (I_t, q_t)$, which consists of RGB images I_t from an overview camera and two wrist cameras, or from any subset of them that the MLLM requests, and the proprioceptive state q_t of the robot. Given the observations $o_{\le t}$ and the earlier actions $a_{<t}$ of the episode, the MLLM issues

$$a_t = F\big(\ell_\tau,\, M^{\text{task}},\, M^{\text{work}}_{k-1},\, o_{\le t},\, a_{<t}\big), \tag{1}$$

where ℓ_τ is the task instruction with basic specifications of the robot embodiment, and M^{task} and M^{work}_{k-1} are the task and working memories of Sections 3.2 and 3.3. Each action a_t is a chunk of 1 to 6 absolute waypoints, each setting an end-effector pose and a gripper target for each arm and, on the mobile robot of EBench, a base pose. Inverse kinematics converts the end-effector poses into joint commands. An episode ends when the evaluator registers success, when the step limit of the task is reached, or earlier when the MLLM ends its turn. The observations and actions of episode k form its trajectory $\xi_k = (o_1, a_1, o_2, a_2, \dots)$. By comparison, a *harness with policy* hands control to a post-trained policy for a bounded number of physics steps and receives only the end state, so ξ_k omits the actions that the policy executes.

Information protocol. On both benchmarks, we restrict the agent to the information that a physical deployment provides. During execution, the agent observes the scene solely through RGB images and robot proprioception $o_t = (I_t, q_t)$, matching the inputs of a post-trained VLA or WAM. Although recent evaluation guidelines for robot-use agents allow camera intrinsics [41], we exclude privileged information such as depth, camera parameters, or object states used by prior systems [22, 23, 29]. On a physical robot, such information would come from external estimation modules that are sensitive to perception errors. Restricting observations to RGB views and proprioception therefore reflects more faithfully how the agent adapts to the robot embodiment and refines

its execution in the environment. Beyond o_t, the MLLM receives only the instruction ℓ_τ, the two memories, and the training demonstration D_τ of Section 3.2. Finally, like post-trained policies, the agent acts within the native per-task step limits of each benchmark and receives no intermediate progress signals.

3.2 Task Memory from a Demonstration

The *task memory* M^{task} records the stage order and the visual completion criterion that a single demonstration conveys reliably. Before the first episode, an annotator instance of the same MLLM reviews the recorded actions and camera frames of the demonstration D_τ and writes M^{task} once. M^{task} lists the stages of the task in order, stating for each stage the operation and the arm that performs it, and ends with the visual state of the completed task. It contains no coordinates, so that it applies to layouts other than the demonstrated one. Because the final stage specifies the completed state, M^{task} gives the MLLM an explicit stopping criterion before any execution experience exists, which we term *terminal-state anchoring*. The MLLM reads M^{task} in every episode and can revisit D_τ during execution. Tasks without a demonstration, the eight Open tasks of RoboDojo, run without M^{task}. Appendix A.2 provides the annotation prompt and the tools for reviewing the demonstration.

3.3 Working Memory via Trajectory Reflection

The *working memory* M^{work} carries execution experience across the episodes of a task through compact text rewrites. After episode k, the same MLLM F updates the working memory in a single reflection turn without robot actions:

$$M_k^{\text{work}} = F\big(M_{k-1}^{\text{work}},\, \xi_k,\, r_k\big), \qquad |M_k^{\text{work}}| \le B, \tag{2}$$

where ξ_k is the executed trajectory and r_k is the terminal outcome, the success and score reported by the benchmark evaluator as an episode-level reward. In each update, the MLLM may replace M_{k-1}^{work} with a full rewrite within a size budget B of 32 KB, so the MLLM itself decides what the bounded memory keeps. Replacing the memory in full allows the MLLM to revise or discard outdated content while keeping M_k^{work} far shorter than the cumulative history of the chain. By analyzing the executed commands and their observed consequences in ξ_k, particularly the causes of failures, the MLLM distills concise lessons on task procedure, completion criteria, and error recovery, accompanied by representative commands and outcomes from earlier attempts as supporting evidence. To discourage path dependence on the MLLM's own exploration trajectories, the description of the memory-writing tool asks it to treat measured coordinates as scene-specific evidence instead of commands to replay in another scene. Appendix A.3 gives the reflection prompt and the memory tools, and Appendix B.2 presents excerpts of working memories.

3.4 Online Adaptation and Deployment

During online adaptation, the agent executes the episodes of a task sequentially along an adaptation *chain*, starting with the working memory M_0^{work} empty. Under the *train–test protocol*, online adaptation runs exclusively on training scenes or layouts, after which the working memory M_K^{work} at the end of the chain is fixed. At deployment, M_K^{work} takes the place of M_{k-1}^{work} in (1) in every new scene without further updates, so no information from deployment scenes can leak into the memory. Appendix B.3 analyzes the dynamics of the adaptation chains.

4 Experiments

We evaluate SimplEvo on two public manipulation benchmarks against their leaderboard entries and against GPT-6-Astra without working memory, with adaptation confined to training scenes.

4.1 Setup

Benchmarks and generalization. The test episodes of EBench [15] and RoboDojo [6] evaluate the working memory under variations in scene conditions. EBench comprises 26 tasks of a simulated ARX Lift 2 dual-arm mobile robot in Isaac Sim [14], and its 510 test scenes perturb the manipulated

objects, the background, the instruction, or several of them at once. Its seven single-layout tasks keep one object layout across training and test episodes while varying objects, backgrounds, and instructions. Its 19 multi-layout tasks also vary the layout, so a test layout may be unseen or already seen in training. Fifteen of these 19 tasks include both seen and unseen test layouts. RoboDojo comprises 42 dual-arm tabletop tasks in five capability dimensions, each tested on 50 layouts of initial objects and their poses. On its 12 Generalization tasks, half of the test layouts randomize the background, the lighting, the target objects, and the clutter. Following the official protocols, we report SR and the stage-completion score, both on a 0–100 scale.

Train–test protocol. Per task, adaptation uses up to 15 EBench training episodes or 15 RoboDojo training episodes, then fixes the working memory for testing. On EBench, adaptation stops early after three consecutive successes, as Appendix B.3 details. On RoboDojo Generalization tasks, training includes both standard and randomized configurations. The task memory M^{task} comes from one training demonstration per task.

Agent and baselines. We use GPT-6-Astra [40] at high reasoning effort as the MLLM core F in all experiments except the cross-model transfer study. On EBench, we compare against the 12 post-trained VLAs and WAMs on the public leaderboard, as well as GPT-6-Astra under direct control and in harnesses with $\pi_{0.5}$ [2] or InternW0-Δ [36] following Yao et al. [60], all reading the task memory but no working memory. On RoboDojo, we compare against the leading post-trained policies and agents on the public leaderboard, the official zero-shot entry of GPT-6-Astra on test layouts, and task memory only on the training layouts used by SimplEvo. We also report tokens per success, the total tokens of an evaluation divided by its successes, and analyze cost in Appendix G.

4.2 Results on EBench

With its working memory fixed, SimplEvo reaches 81.7% SR on the 510 test episodes of EBench and ranks first in every task group of Table 1, using the leaderboard snapshot of 29 October 2026, ahead of PICA-WAM [72], the best post-trained policy, at 61.4% and the best harness with policy at 56.0%. The largest gains arise on high-precision, tabletop, and long-horizon tasks. On high-precision tasks, direct control without working memory succeeds in 11.3% of the episodes, and neither harness with policy exceeds 12.5%. With working memory, the same direct control reaches 82.5%, 2.3 times the 36.3% of PICA-WAM, which shows that the waypoints of the MLLM are precise enough for these tasks once the corrections of earlier attempts persist. On long-horizon tasks, direct control drops to 27.3% from 73.2% on short-horizon ones, while SimplEvo reaches 76.3%, above even the short-horizon SR of direct control. The task memory fixes the stage sequence, and the working memory adds to each stage the completion checks and recovery steps verified in earlier episodes. On bottle collection, SimplEvo reaches 45.0% SR, while none of the 12 leaderboard policies succeeds. It also reaches 90.0% on coffee-bean collection and 95.0% on shop checkout, where direct control without working memory has no successes. At test time, SimplEvo spends 2.76M tokens per success, compared with 8.12M for direct control, 3.46M for the harness with $\pi_{0.5}$, and 3.99M for the harness with InternW0-Δ, as Appendix G reports.

4.3 Results on RoboDojo

After adapting on the 15 training layouts of each task, SimplEvo ranks first among the RoboDojo leaderboard entries of Table 2 with 69.63% SR and a score of 75.32 on the 50 test layouts per task. It also has the highest SR and score when the Open dimension is excluded. Table 3 shows two effects of the working memory. First, reflection distills experience into working memory on the eight Open tasks, which provide no demonstration or task memory. On the training layouts, SR rises from 32.50% without working memory to 70.00% during adaptation. After adaptation, the fixed working memory reaches 77.50% on the test layouts. Second, the fixed working memory carries training experience into test episodes. Test SR exceeds the 58.53% observed during online adaptation, and tokens per success are 2.23M on test layouts, compared with 4.12M for task memory alone on training layouts. Appendix D.1 compares agentic systems outside the leaderboard.

Table 1: SR (%) and score on the 510 test episodes of EBench for the post-trained policies on the public leaderboard (29 October 2026) and for GPT-6-Astra without working memory and with SimplEvo. Bold and underline mark the best and second-best values per column.

	Overall		Precision			Mobility		Horizon	
System	**SR**	**Score**	**Low**	**Med.**	**High**	**Mobile**	**Tabletop**	**Short**	**Long**
Post-trained policies									
X-VLA [70]	23.7	35.8	35.7	13.3	2.5	29.3	8.6	45.9	7.4
Fast-WAM [63]	25.6	37.1	35.7	19.0	3.8	34.0	2.9	46.4	10.4
Cosmos3-Edge [39]	29.3	42.5	38.6	20.8	13.8	33.2	18.6	39.1	22.1
GigaBrain-0.7 [17]	33.3	46.0	35.4	33.8	25.0	31.6	37.9	40.0	28.3
π_0 [3]	33.7	47.5	39.0	30.0	22.5	34.0	32.9	40.5	28.8
InternVLA-A1.5 [34]	34.2	46.8	44.6	26.0	13.8	42.3	12.1	46.8	24.9
$\pi_{0.5}$ [2]	41.4	54.4	49.6	39.0	17.5	45.9	29.3	52.3	33.4
Qwen-RobotManip [62]	45.6	60.8	50.6	50.2	18.8	43.9	<u>50.0</u>	55.9	38.0
InternW0-Δ [36]	49.2	66.0	58.7	49.2	16.3	55.0	33.6	59.1	42.0
Vela [27]	49.7	65.5	59.5	45.4	23.8	51.7	44.3	59.5	42.4
OpenWAM-α [55]	55.3	70.1	67.4	45.6	32.5	60.2	42.1	65.9	47.6
PICA-WAM [72]	<u>61.4</u>	<u>74.9</u>	<u>77.5</u>	45.8	<u>36.3</u>	<u>67.5</u>	45.0	68.6	<u>56.1</u>
Agents based on GPT-6-Astra									
Direct control [60]	46.7	65.4	60.6	40.2	11.3	56.6	20.0	73.2	27.3
Harness with $\pi_{0.5}$ [60]	53.0	69.2	68.9	47.9	7.5	61.0	31.4	70.5	40.2
Harness with InternW0-Δ [60]	56.0	74.4	68.2	<u>56.5</u>	12.5	64.3	33.6	<u>77.3</u>	40.4
SimplEvo (ours)	**81.7**	**91.2**	**90.1**	**66.7**	**82.5**	**83.2**	**77.9**	**89.1**	**76.3**

Table 2: SR (%) and score on the 42 tasks of RoboDojo, over 50 layouts per task, for entries on the public leaderboard (2 November 2026) and for SimplEvo on its test layouts. The w/o Open columns average the four dimensions other than Open, as on the leaderboard. Bold and underline mark the best and second-best values.

		Overall		w/o Open		Capability SR (%)				
System	**Class**	**SR**	**Score**	**SR**	**Score**	**Gen.**	**Prec.**	**Long**	**Mem.**	**Open**
Post-trained policies										
$\pi_{0.5}$ [2]	VLA	6.93	11.44	8.25	13.80	8.17	5.50	14.67	4.67	1.67
Awomo-0.5 [1]	WAM	29.64	35.34	31.33	38.15	23.84	33.50	26.75	41.22	22.92
VPP2 [19]	WAM	32.26	39.26	39.31	47.93	29.50	35.17	37.00	55.56	4.08
WALL-OSS-1.0-lite [64]	VLA	34.97	42.59	38.82	48.05	37.94	29.08	32.25	56.00	19.58
Agents										
GPT-6-Astra, zero-shot [66]	Agent	22.48	28.97	20.36	27.63	30.50	4.00	8.25	38.67	31.00
WorldScape-R^{2}SI [35]	Agent + WAM	39.35	46.58	37.31	45.93	35.83	15.50	34.25	63.67	47.50
HexaShell-Preview [13]	Agent	46.85	53.18	44.75	51.98	34.50	32.00	31.50	<u>81.00</u>	55.25
PyRUA-Lean [47]	Agent	50.35	55.26	48.37	53.93	<u>43.17</u>	38.25	43.75	68.33	<u>58.25</u>
GO-3 Nano Preview	Agent + WAM	<u>56.42</u>	<u>61.90</u>	<u>56.71</u>	<u>61.95</u>	33.66	**76.00**	<u>53.50</u>	63.67	55.25
SimplEvo (ours)	Agent	**69.63**	**75.32**	**67.67**	**74.09**	**52.00**	<u>64.50</u>	**58.50**	**95.67**	**77.50**

5 Analysis

5.1 Design Choices for Memory and Reflection

Text versus visual memory. On online chains of ten validation scenes for each of the four high-precision EBench tasks, the task memory stays fixed and only the working-memory representation varies. Each chain starts from an empty working memory, so its SR includes the first attempts of adaptation, unlike the fixed working memory of Table 1. Under direct control, Table 4 shows that the text working memory more than triples the SR of task memory alone, from 15.0% to 47.5%. Adding a visual replay of earlier attempts, which archives their camera frames, to the text working memory leaves the SR unchanged, and the MLLM queries the replay in only 3 of the 40 episodes. Maintaining a single continuous conversation across the chain, following the visual history design of VISTA [20], reaches a similar SR with a slightly higher score. The text working memory uses

Table 3: SR (%) and score of GPT-6-Astra on the 42 tasks of RoboDojo: task memory only and online adaptation on the 15 training layouts per task, and the fixed working memory of SimplEvo on the 50 test layouts per task. Bold marks the best value in each column.

		Overall		w/o Open		Open
Configuration	**Layouts (Episodes)**	**SR**	**Score**	**SR**	**Score**	**SR**
Task memory only	15 training (630)	36.37	43.69	37.34	45.78	32.50
SimplEvo, online	15 training (630)	58.53	65.04	55.66	63.15	70.00
SimplEvo, fixed working memory	50 test (2,100)	**69.63**	**75.32**	**67.67**	**74.09**	**77.50**

Table 4: Memory designs of GPT-6-Astra under direct control on one online chain of ten validation scenes per high-precision EBench task: SR (%) per task and overall, mean score, tokens per episode (M), and minutes of control per episode at equal simulator speed (model time plus 0.5 s per physics step). The visual-replay row includes text working memory, and VISTA history keeps the whole chain in one conversation. Bold marks the best value in each column.

Memory	**Coffee**	**Gear**	**Peg**	**Nut**	**SR**	**Score**	**Tokens**	**Minutes**
Task memory only	0.0	40.0	10.0	10.0	15.0	40.5	3.51	29.5
+ text working memory (SimplEvo)	20.0	70.0	**90.0**	10.0	**47.5**	64.1	**2.86**	**23.0**
Text working memory + visual replay	**30.0**	70.0	70.0	20.0	**47.5**	64.5	3.47	27.2
VISTA history [20]	20.0	**90.0**	40.0	**30.0**	45.0	**68.0**	6.10	31.4

the fewest tokens per episode and the shortest control time, with an SR similar to that of the visual memory designs. Appendix E.1 details the protocol of this design study.

Outcome feedback. The working memory retains most of its gain without terminal outcome feedback. Eight tasks test this signal: the four high-precision tasks and four multi-layout tasks, namely dishwasher loading, the fruit task, utensil placement, and dish placement. When the terminal outcome r_k is withheld during online adaptation along their 20-scene chains, the overall SR decreases from 73.8% to 65.0%. The high-precision chains keep 52.5% SR in both conditions. On the final five episodes of the four multi-layout chains, where task memory alone also runs on the same scenes, working memory without outcome feedback reaches 80.0% SR, compared with 35.0% for task memory alone and 95.0% with outcome feedback, thereby retaining three quarters of the improvement through self-assessment of ξ_k alone, as detailed in Appendix E.2.

Insight 1. A bounded text working memory suffices: it matches the SR of visual memory at lower cost and keeps most of its gain without outcome feedback.

5.2 Memory Pays Off Under Direct Control

Table 5 shows a larger SR gain from the proposed memory mechanism under direct control than in the harness with $\pi_{0.5}$ on the same chains. Under direct control, peg insertion rises from one success in ten scenes to nine. In the harness with $\pi_{0.5}$, adding text working memory moves the SR from 22.5% to 25.0%, and adding the visual replay lowers it to 12.5%.

Neither exposing the motion of the policy nor adding an external critic closes this gap. When the harness reports the executed end-effector trajectory of $\pi_{0.5}$ after each call and the MLLM reflects on it, the SR of the harness falls to 17.5%, as Appendix E.3 details.

In an independent experiment, a critic-driven RSI loop lets an instance of GPT-6-Astra inspect training episodes over five rounds and revise a playbook and the harness with $\pi_{0.5}$. The loop raises the SR of this harness only from 44.4% with an empty playbook to 46.7% on 90 paired test episodes of 18 tasks. On the same episodes, SimplEvo under direct control reaches 76.7% SR and matches or exceeds the critic-driven loop on 17 of the 18 tasks, with per-task results in Appendix E.4. Under direct control, reflection revises the low-level commands that the MLLM issues itself, whereas in the harness with $\pi_{0.5}$ it can only update the guidance for calling a fixed policy.

Table 5: Working memory under the two agent harnesses of GPT-6-Astra on the chains and with the columns of Table 4. The visual-replay row includes text working memory, and the trajectory row extends it with the executed motion of $\pi_{0.5}$, as Appendix E.3 describes. Bold marks the best value within each harness.

Memory	Coffee	Gear	Peg	Nut	SR	Score	Tokens	Minutes
Harness with $\pi_{0.5}$								
Task memory only	0.0	**60.0**	10.0	**20.0**	22.5	41.3	**1.73**	27.9
+ text working memory	0.0	**60.0**	**30.0**	10.0	**25.0**	**51.6**	2.88	**26.5**
Text working memory + visual replay	**10.0**	20.0	10.0	10.0	12.5	40.7	2.57	27.0
Text + $\pi_{0.5}$ trajectory	**10.0**	40.0	0.0	**20.0**	17.5	43.2	2.78	28.0
Direct control								
Task memory only	0.0	40.0	10.0	**10.0**	15.0	40.5	3.51	29.5
+ text working memory (SimplEvo)	**20.0**	**70.0**	**90.0**	**10.0**	**47.5**	**64.1**	**2.86**	**23.0**

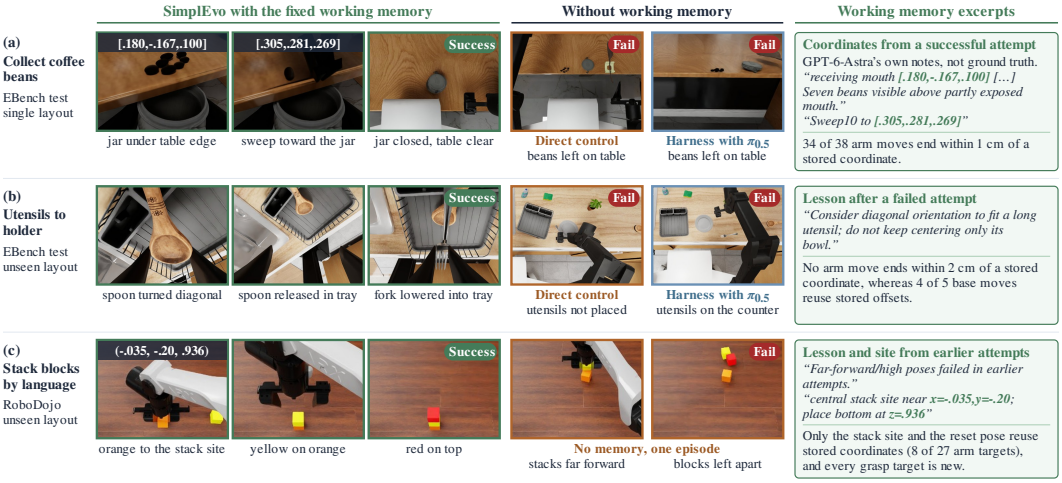

Figure 3: Test episodes with the fixed working memory of SimplEvo (green) and without working memory on the same scenes, with verbatim excerpts of the memory read in each SimplEvo episode ([. . .] marks omissions). Each row shows a success of SimplEvo on a scene where the other configurations fail. Over all test episodes of these tasks, SimplEvo succeeds in 18 of 20 in (a), 12 of 20 in (b), and 49 of the 50 test layouts in (c), compared with none in (a) and 2 in (b) for direct control without working memory, and none and 3 for the harness with $\pi_{0.5}$. Dark labels give commanded positions in meters equal to coordinates in the working memory, which GPT-6-Astra wrote from its own commands and proprioception in training episodes without access to ground-truth object positions. Baseline frames in (a) and (b) come from Table 1 and in (c) from Appendix E.5.

Insight 2. Within the compared frameworks, the proposed memory evolution improves direct control more than the harness with policy.

5.3 How the Memory Generalizes to Unseen Scenes

With its memories fixed, SimplEvo sustains its SR on unseen layouts and randomized configurations, whereas the SR of every other system in Table 6 drops on them. On the 15 multi-layout EBench tasks with both kinds of test episodes, SimplEvo succeeds more often on unseen layouts than on seen ones, and each configuration of GPT-6-Astra without working memory drops by 2.8 to 10.7 points. On the 12 Generalization tasks of RoboDojo, SimplEvo keeps 52.00% SR in both configurations, whereas the zero-shot entry of the same MLLM drops from 32.67% to 28.33% and the two strongest post-trained policies lose 15.00 and 33.88 points. Matched comparisons that rerun task memory alone on the same test scenes isolate the contribution of the working memory. With the task memory M^{task} held fixed, adding the fixed working memory raises the SR from 67.4% to 85.3% on the last five

Table 6: SR (%) by layout overlap on EBench and scene configuration on RoboDojo. Seen EBench layouts occur in at least one completed training episode of SimplEvo. The same partition applies to every configuration of GPT-6-Astra. On RoboDojo, each Generalization task pairs a standard configuration with a randomized one, the other rows are the entries of the public leaderboard (2 November 2026), and the 15 training layouts of SimplEvo include standard and randomized configurations. SR is averaged within each task and then across tasks, and Δ is the change from the first to the second column. Class marks vision-language-action models (VLA), world action models (WAM), and MLLM agents (Agent), and Agent + VLA or Agent + WAM denotes an agent that calls such a policy. Bold and underline mark the best and second-best values within each benchmark.

System	Class	Seen / standard	Unseen / randomized	Δ
EBench, 15 multi-layout tasks: seen and unseen layouts				
Direct control [60]	Agent	73.8	63.1	−10.7
Harness with $\pi_{0.5}$ [60]	Agent + VLA	<u>77.2</u>	69.6	−7.6
Harness with InternW0-Δ [60]	Agent + WAM	74.5	<u>71.7</u>	<u>−2.8</u>
SimplEvo (ours)	Agent	**83.1**	**87.5**	**+4.4**
RoboDojo, 12 Generalization tasks: standard and randomized configurations				
Awomo-0.5 [1]	WAM	30.89	16.78	−14.11
VPP2 [19]	WAM	<u>46.44</u>	12.56	−33.88
WALL-OSS-1.0-lite [64]	VLA	45.44	30.44	−15.00
GPT-6-Astra, zero-shot [66]	Agent	32.67	28.33	−4.34
WorldScape-R^{2}SI [35]	Agent + WAM	40.33	31.33	−9.00
HexaShell-Preview [13]	Agent	38.00	31.00	−7.00
PyRUA-Lean [47]	Agent	45.33	<u>41.00</u>	<u>−4.33</u>
GO-3 Nano Preview	Agent + WAM	43.00	24.33	−18.67
SimplEvo (ours)	Agent	**52.00**	**52.00**	**0.00**

test episodes of each of the 19 multi-layout EBench tasks and from 40.83% to 66.11% on three test layouts of each of the 42 RoboDojo tasks, as Appendix E.5 details.

The working memory pairs distilled lessons with evidence from earlier attempts, as in Figure 3a, and the episode prompt of Appendix A presents these notes as hypotheses to verify against the current views. On an unseen layout, as in Figure 3b and c, the MLLM locates the objects in the current views and follows the procedural, completion, and recovery lessons, so the commanded positions of successful episodes rarely coincide with stored ones, and in the median episode only 11% of the moving waypoints come within 1 cm of any stored position. Appendices E.6 and H analyze the commanded waypoints and present further case studies.

Insight 3. Lessons about task invariants, such as procedure, completion checks, and error recovery, hold across layouts and scene conditions, so a memory built from them generalizes beyond the scenes that produced it.

5.4 Memory Transfers Across Models

Read without modification, the fixed working memory of GPT-6-Astra improves other MLLMs under direct control on the Mixed generalization setting of EBench across all 26 tasks, as shown in Table 7. Each MLLM attempts every scene of this dimension in two independent sessions, once with task memory alone and once with the memory that GPT-6-Astra uses in Table 1. With this memory, Gemini 3.8 Flash more than doubles its SR, DeepSeek-V4.1-Flash succeeds on scenes it almost never solves with task memory alone, and GPT-6-Astra itself improves as well. Appendix F breaks these gains down by task layout and extends the study to four MLLMs along the adaptation chains of two tasks. Transfer remains limited when the working memory omits low-level control behaviors. Over 24 paired episodes of eight tasks, both with and without the memory, DeepSeek-V4.1-Flash sets the gripper target near the measured width of the object in 19% of its lifting waypoints and thus does not squeeze the object. GPT-6-Astra does so in only 5% of its lifting waypoints because

Table 7: Transfer of the working memory of GPT-6-Astra across MLLMs, measured on the 130 test episodes of the Mixed generalization setting of EBench, five per task across all 26 tasks: SR (%) and mean score with task memory only and with the working memory of GPT-6-Astra read without modification. The working-memory entry of GPT-6-Astra comes from Table 1. Bold marks the higher value for each model.

	Task memory only		Task + working memory of GPT-6-Astra	
Model	**SR**	**Score**	**SR**	**Score**
Other MLLMs				
Gemini 3.8 Flash	23.8	37.8	**53.8**	**71.7**
DeepSeek-V4.1-Flash	0.8	4.6	**13.1**	**26.5**
MLLM that wrote the working memory				
GPT-6-Astra	60.8	76.8	**81.5**	**89.3**

it consistently closes the gripper fully on a lifted object, and its working memory therefore never states this low-level control habit.

Insight 4. Text working memory carries the explicit execution lessons of a stronger MLLM to weaker MLLMs without weight updates, whereas low-level control habits that the memory leaves unstated do not transfer.

6 Conclusion

Although embodied agents increasingly surround frontier MLLMs with perception pipelines, harnesses with policy, and critic loops, our results show that a frontier MLLM under direct control already achieves strong closed-loop manipulation once its within-episode corrections persist across episodes. SimplEvo retains these corrections in two self-written text memories: a coordinate-free task memory distilled once from one training demonstration, and a bounded working memory rewritten after each training episode. Under the strict train–test protocol, with RGB views and proprioception as its only observations, SimplEvo raises GPT-6-Astra from 46.7% to 81.7% SR on EBench and leads the RoboDojo leaderboard. Our analyses yield four takeaways for embodied agent design. (1) A bounded text memory distilled from the trajectories of the agent itself matches the SR of visual memory at lower cost and retains most of the improvement without outcome feedback. (2) The proposed memory mechanism improves direct control more than the compared harness with policy. (3) Lessons on task invariants, applied to the objects that the MLLM locates in the current views, hold on unseen layouts and under randomized configurations. (4) A text memory written by one MLLM also improves other MLLMs that read it without modification, though only for the behaviors that it states explicitly.

Limitations and future directions. These findings point to three directions for future work. First, *verified memory updates* could validate each candidate rewrite before it enters the working memory and thus filter out spurious lessons. Second, *object-centric, behavior-explicit memory* could anchor recorded actions to visual features and state the low-level control habits that GPT-6-Astra leaves implicit, so that recorded evidence carries across workspace geometries and the memory serves weaker MLLMs. Third, both benchmarks pause the simulator during MLLM deliberation and end episodes automatically upon completion, and each episode costs millions of tokens. These conditions motivate *asynchronous, token-efficient deployment on physical robots* with autonomous completion detection [41].

References

[1] Awomo-WAM Team. Awomo-0.5 technical report. https://github.com/Awomo-WestlakeDI/Awomo-0.5/blob/main/Awomo_0_5.pdf, 2026. Technical report.

[2] Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pp. 17–40. PMLR, 2025.

[3] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, Laura Smith, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: A vision-language-action flow model for general robot control. In *Proceedings of Robotics: Science and Systems*, 2025. doi: 10.15607/RSS.2025.XXI.010.

[4] Konstantinos Bousmalis, Giulia Vezzani, Dushyant Rao, Coline Devin, Alex X. Lee, Maria Bauza, Todor Davchev, Yuxiang Zhou, Agrim Gupta, Akhil Raju, Antoine Laurens, Claudio Fantacci, Valentin Dalibard, Martina Zambelli, Murilo Martins, Rugile Pevceviciute, Michiel Blokzijl, Misha Denil, Nathan Batchelor, Thomas Lampe, Emilio Parisotto, Konrad Żołna, Scott Reed, Sergio Gómez Colmenarejo, Jon Scholz, Abbas Abdolmaleki, Oliver Groth, Jean-Baptiste Regli, Oleg Sushkov, Tom Rothörl, José Enrique Chen, Yusuf Aytar, Dave Barker, Joy Ortiz, Martin Riedmiller, Jost Tobias Springenberg, Raia Hadsell, Francesco Nori, and Nicolas Heess. RoboCat: A self-improving generalist agent for robotic manipulation. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL https://openreview.net/forum?id=vsCpILiWHu. arXiv:2306.11706.

[5] Tianxing Chen, Zanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, et al. RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.

[6] Tianxing Chen, Yue Chen, Zixuan Li, Junyuan Tang, Kailun Su, Haoran Lu, Weijie Wan, Baijun Chen, Songling Liu, Haowen Yan, Honghao Su, Zhiyang Dou, Kaixuan Wang, Dandan Zhang, Yunze Liu, Yan Qin, Qiwei Liang, Qiwei Wu, Zijian Lin, Wenwei Lin, Yuran Wang, Minghua He, Tianshu Wu, Ruihai Wu, Jingquan Zhou, Kai-Chong Lei, Haibao Yu, Yuanfeng Ji, Weiyang Jin, Guanyu Lin, Xiaofan Li, Qi Xiong, Renjing Xu, Zhongyu Li, Wenhao Chai, Enze Xie, Ziwei Wang, Yao Mu, Hao Dong, Wojciech Matusik, Mingyu Ding, Wenbo Ding, Ping Luo, and Masayoshi Tomizuka. RoboDojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies. *arXiv preprint arXiv:2607.04434*, 2026.

[7] Yanzhe Chen, Zechen Bai, Zhijun Cao, Wenzheng Zeng, Kevin Qinghong Lin, Yiqi Lin, Guoqiang Liang, Kevin Yuchen Ma, Qiming Huang, and Mike Zheng Shou. Show-Harness: Just a VLM agent can play robots. *arXiv preprint arXiv:2609.10522*, 2026.

[8] An-Chieh Cheng, Isabella Liu, Edmund Bu, Johan Bjorck, Hongxu Yin, Zhengyi Luo, Jan Kautz, Linxi Fan, Yuke Zhu, and Sifei Liu. ASENA: Self-evolving agents for embodied navigation. *arXiv preprint arXiv:2609.39207*, 2026.

[9] Dongzhou Cheng, Taoran Yi, Ye Fang, Xingwu Zhang, Fan Feng, Yixuan Li, Gengxiong Zhuang, Rongze Wang, Shuai Yang, Wei Song, Weizhi Xue, Minyan Wu, Jie Gui, Jiaqi Wang, and Tong Wu. In-context robot learning with VLM agents. *arXiv preprint arXiv:2609.19138*, 2026.

[10] Xin Ding, Liang Mi, Mingzhe Huang, Zixuan Wang, Chao Zhang, Zixu Hao, Fu Chen, Xiangyu Li, Yikai Zheng, Yaoyu Guo, Weijun Wang, Kun Li, Hao Wu, Yunxin Liu, and Ting Cao. Zetta ζ: An efficient closed-loop embodied harness for self-evolving physical intelligence. *arXiv preprint arXiv:2608.16590*, 2026.

[11] Karim Elmaaroufi, Justin Svegliato, Sarunas Kalade, Graham Schelle, Sanjit A. Seshia, and Matei Zaharia. RHO: Your coding agent is secretly a roboticist. *arXiv preprint arXiv:2606.16458*, 2026.

[12] Letian Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, Guanzhi Wang, Dantong Niu, Fei-Fei Li, Guanya Shi, Jiajun Wu, Shankar Sastry, Yuke Zhu, Ken Goldberg, and Linxi Fan. CaP-X: A framework for benchmarking and improving coding agents for robot manipulation. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2603.22435.

[13] Hongcheng Gao, Jianshu Zeng, Alex Zhou, Shijia Ge, Xuan Shangguan, Yazhe Wang, Yexing Wan, Zelin Zheng, Jay Zhu, Xiping Li, Di Wu, Yongli Wang, Lingyu He, Zhiqi Jia, Sihang Wu, and Xiao He. HexaShell: A universal harness for robot coding agents in simulation and the real world. *arXiv preprint arXiv:2610.23479*, 2026.

[14] Ning Gao, Yilun Chen, Shuai Yang, Xinyi Chen, Yang Tian, Hao Li, Haifeng Huang, Hanqing Wang, Tai Wang, and Jiangmiao Pang. GenManip: LLM-driven simulation for generalizable instruction-following manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12187–12198, 2025. doi: 10.1109/CVPR52734.2025.01138.

[15] Ning Gao, Jinliang Zheng, Xing Gao, Haoxiang Ma, Hanqing Wang, Yukai Wang, Jiantong Chen, Zanxin Chen, Shujie Zhang, Mingda Jia, Xuekun Jiang, Zihou Zhu, Xinyu Li, Shuai Wang, Hao Li, Wenzhe Cai, Yuqiang Yang, Xudong Xu, Zhaoyang Lyu, Yao Mu, Tai Wang, Jiangmiao Pang, Jia Zeng, Weinan Zhang, and Chunhua Shen. EBench: Elemental diagnosis of generalist mobile manipulation policies. *arXiv preprint arXiv:2606.18239*, 2026.

[16] Yonatan Gideoni, Sebastian Risi, and Yarin Gal. Simple baselines are competitive with code evolution. *arXiv preprint arXiv:2602.16805*, 2026.

[17] GigaBrain Team, Angen Ye, Axiang Sun, Can Jin, Chenxi Cheng, Chong Shi, Dengke Shang, Dingqian Zhang, Guan Huang, Guangqiang Wang, Guangqing Ding, Guo Li, Hangcong Li, Hengyu Zhong, Hongtao Lu, Jianbo Qin, Jiming Mao, Jing Zhu, Jindi Lv, Jingzhi Cui, Junjie Xie, Junyi Bao, Kai Liu, Lei Yuan, Limin Long, Lv Feng, Mingming Yu, Peng Li, Pengfei Yi, Qi Li, Qianli Zhang, Qingfang Li, Qitang Hu, Rui Zhang, Shaoyan Sun, Shibo Sun, Shiying Duan, Tenghui Chen, Tianze Liu, Weijie Ke, Wenyao Xue, Xiaofeng Wang, Xiaoyu Tian, Xinyu Liu, Xinze Chen, Yang Wang, Yankai Wang, Yejun Zeng, Yifan Li, Yifei Nie, Yilong Li, Yilong Liu, Yongchao Feng, Yumeng Wang, Yun Ye, Zhichao Liu, Ziheng He, Zonghai Yang, and Zheng Zhu. GigaBrain-0.7: Scaling embodied foundation models to emergent capabilities with a three-system architecture. *arXiv preprint arXiv:2608.15875*, 2026.

[18] Meng-Hao Guo, Zhe-Han Mo, Jia-Jun Wang, Yi Zhang, Kejin Wang, Yi-Xuan Deng, Jia-Peng Zhang, Yongming Rao, and Shi-Min Hu. Transferring the intelligence of VLMs to robotic control. *arXiv preprint arXiv:2609.22966*, 2026.

[19] Yanjiang Guo, Haodong Yan, Zhide Zhong, Zhongru Zhang, Qingyuan Yang, Qingzhou Lu, Xiaoyu Chen, Yen-Jen Wang, Shuying Deng, Chenghan Yang, Puzhen Yuan, Chenxin Liu, Tun Ban, Xiang Zhu, Yichen Liu, Kun Feng, Haoang Li, and Jianyu Chen. Video prediction policy 2: Predict better, act better. *arXiv preprint arXiv:2610.10270*, 2026.

[20] Qiushi Han, Keya Hu, Linlu Qiu, Cathy Wu, and Kaiming He. VISTA: A visual harness for reasoning in an interactive world. *arXiv preprint arXiv:2610.02200*, 2026.

[21] Hengyuan Hu, Jensen Gao, Priya Sundaresan, Satvik Sharma, Jeannette Bohg, and Dorsa Sadigh. VIA: Visual interface agent for robot control. *arXiv preprint arXiv:2607.11119*, 2026.

[22] Kairui Hu, Siyuan Hu, Fangzhou Hong, Zhaoxi Chen, and Ziwei Liu. Embodied Turing machines: Stateful code for robot recursive self-improvement. *arXiv preprint arXiv:2610.12369*, 2026.

[23] Runhan Huang, Xiangcheng Zhang, Haoyu Chen, Yitong Li, Yen-Jen Wang, Bohan Lyu, Haotian Ye, Chuye Hong, Yixuan Wang, Chloe Su, Haoyu Xiong, Yangzhen Wu, Enze Xie, Travis

Manderson, Stefano Ermon, Pieter Abbeel, Antonio Torralba, Dawn Song, Weirui Ye, and Yilun Du. Scaling online exploration for robot agents. *arXiv preprint arXiv:2610.22496*, 2026.

[24] Mengzhao Jia, Yang Lin, Xixin Zhang, Zhihan Zhang, Xiaobai Liu, and Meng Jiang. Agent as policy for robotic manipulation. *arXiv preprint arXiv:2609.12541*, 2026.

[25] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[26] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 4015–4026, 2023.

[27] Yifan Li, Jiaxu Wang, Dongming Wu, Yicheng Jiang, Ryan Ji, Xiangyu Yue, and Yanwei Fu. Vela: Scaling vision-language-action models with adaptive action curve parametrization. *arXiv preprint arXiv:2610.05230*, 2026. URL https://arxiv.org/abs/2610.05230.

[28] Zexi Li, Yehang Zhang, Haojian Huang, Bohan Zhou, Wenqian Li, Chenxu Wang, Yifan Chang, Yangkai Wei, Tianyi Zhang, Ying-Cong Chen, Kaiwen Zhou, Yinchuan Li, and James Cheng. RACaP: Agentic reasoning, acting, and coding as policies for evolvable robot learning. *arXiv preprint arXiv:2609.29394*, 2026.

[29] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as Policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023. doi: 10.1109/ICRA48891.2023.10160591.

[30] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pp. 44776–44791, 2023.

[31] Fangcheng Liu, Yeqing Shen, Anda Cheng, Weishi Mi, Chao Tang, Chenyuan Liu, Yushun Xiang, Tingguang Li, Yong-Lu Li, and Yehui Tang. RoboICL: Embodied in-context learning with GPT-6 Astra. *arXiv preprint arXiv:2609.34261*, 2026.

[32] Zeyu Lou, Yanhong Zeng, Yong Wang, and Chenyang Si. Know your body: A harness for direct and self-improving robot control with VLMs. *arXiv preprint arXiv:2609.28530*, 2026.

[33] Runyu Lu, Yubo Wu, Ethan Kou, Haotian Lin, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzhen Xu, Guanya Shi, Ken Goldberg, Ang Chen, Mosharaf Chowdhury, Jan Kautz, Yuke Zhu, Linxi "Jim" Fan, and Guanzhi Wang. ASPIRE: Agentic skills discovery for robotics. *arXiv preprint arXiv:2607.00272*, 2026.

[34] Haoxiang Ma, Junhao Cai, Xiaoxu Xu, Hao Li, Yuyin Yang, Yang Tian, Jiafei Cao, Hongrui Zhu, Zherui Qiu, Zhaxizhuoma, Yuqiang Yang, Jiaqi Peng, Xueyuan Wei, Yangkun Zhu, Jiahao Jiang, Xing Gao, Hanqing Wang, Feng Yuan, Kailin Li, Xueyue Zhu, Tai Wang, Yan Ding, Jiangmiao Pang, Jia Zeng, Jingjing Zhang, Bowen Zhou, Yao Mu, Chunhua Shen, and Weinan Zhang. InternVLA-A1.5: Unifying understanding, latent foresight, and action for compositional generalization. *arXiv preprint arXiv:2607.04988*, 2026.

[35] Manifold AI Team. R^2SI: Recursive robot self-improvement via a bi-level agentic system with agent–model co-evolution. https://r2si.worldscape.io/report.pdf, 2026. Technical report.

[36] Xingyu Miao, Zizun Li, Baole Fang, Kaiwen Song, Tenghui Wang, Hanxue Zhang, Yating Wang, Xudong Li, Yuping He, Xueyuan Wei, Chao Gao, Xijie Yang, Yingxiang Xu, Kerui Ren, Wenqi Guo, Jianjun Zhou, Xinzhe Wang, Weiguang Zhao, Ni Yang, Zetao Cai, Yufei Xue, Hengjie Li, Zeyu He, Yuanzhen Zhou, Rong Fu, Jianyang Zhang, Siwei Cui, Fuxian

Huang, Yunsong Zhou, Xing Gao, Yifei Yao, Qiaojun Yu, Kailin Li, Ming Zhou, Mu Huang, Xinyue Li, Wenze Cui, Bingqi Jiang, Xueyue Zhu, Junting Dong, Haoyu Guo, Tao Lu, Mulin Yu, Bowen Zhou, Bin Zhao, Tianfan Xue, Weinan Zhang, and Chunhua Shen. InternW0-Δ: A world action model bridging predictive dynamics and actions with 20K+ hours of open data, 2026. URL https://arxiv.org/abs/2609.31394.

[37] MMLab & PhysicalRSI contributors. Physical RSI 1.0: Recursive self-harness for scaling embodied skills. Project website and open-source software, https://mmlab.hk/research/PhysicalRSI, 2026. Accessed 2026-10-14.

[38] Yao Mu, Tianxing Chen, Zanxin Chen, Shijia Peng, Zhiqian Lan, Zeyu Gao, Zhixuan Liang, Qiaojun Yu, Yude Zou, Mingkun Xu, et al. RoboTwin: Dual-arm robot benchmark with generative digital twins. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 27649–27660, 2025.

[39] NVIDIA. Cosmos 3: Omnimodal world models for physical AI. *arXiv preprint arXiv:2606.02800*, 2026. URL https://arxiv.org/abs/2606.02800.

[40] OpenAI. GPT-6 Astra system card. OpenAI Deployment Safety Hub, September 2026. URL https://deploymentsafety.openai.com/gpt-6-astra.

[41] OpenRUA Team. Behind the RoboDojo boom: Revisiting embodied evaluation of robot-use agents. OpenRUA research blog, November 2026. URL https://openrua.ai/revisiting-rua.

[42] Physical Intelligence, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Jared DiCarlo, Danny Driess, Michael Equi, Adnan Esmail, Yunhao Fang, Chelsea Finn, Catherine Glossop, Thomas Godden, Ivan Goryachev, Lachy Groom, Hunter Hancock, Karol Hausman, Gashon Hussein, Brian Ichter, Szymon Jakubczak, Rowan Jen, Tim Jones, Ben Katz, Liyiming Ke, Chandra Kuchi, Marinda Lamb, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Yao Lu, Vishnu Mano, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Charvi Sharma, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, Will Stoeckle, Alex Swerdlow, James Tanner, Marcel Torne, Quan Vuong, Anna Walling, Haohuan Wang, Blake Williams, Sukwon Yoo, Lili Yu, Ury Zhilinsky, and Zhiyuan Zhou. $\pi^*_{0.6}$: a VLA that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.

[43] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations (ICLR)*, 2025.

[44] SenseNova Joint Research Team, Zhongang Cai, Ziang Cao, Tailai Chen, Han Chong Chua, Linjun Dai, Hanming Deng, Hokin Deng, Haiwen Diao, Chenjing Ding, Weichen Fan, Xiao Fu, Zerui Gong, Chenyang Gu, Zexin He, Zhi Hou, Bo Li, Qinyue Li, Quanzhou Li, Xiaowan Li, Zhichao Liao, Dahua Lin, Zilong Lin, Bo Liu, Chunxiao Liu, Pengfei Liu, Tianqi Liu, Ziwei Liu, Chen Change Loy, Lewei Lu, Jiapeng Luo, Yuexiao Ma, Liang Pan, Xingang Pan, Oscar Qian, Ziyi Shang, Yuyao Shao, Dawei Su, Dacheng Tao, Qingyi Tao, Quan Wang, Ruisi Wang, Xiaogang Wang, Ziyi Wang, Beichen Wen, Donghao Wu, Jiaxu Wu, Silei Wu, Xiaoming Wu, Zhonghua Wu, Hongcan Xiao, Weiye Xu, Atsushi Yamashita, Lei Yang, Zhe Yang, Baiqiao Yin, Wanqi Yin, Hanfei Zhao, Tianbo Zhong, Long Zhuo, and Jing Zuo. Has GPT-6-Astra achieved embodied intelligence? *arXiv preprint arXiv:2610.22471*, 2026.

[45] Zayx Shawn. PACE: Anytime-valid acceptance tests for self-evolving agents. *arXiv preprint arXiv:2606.08106*, 2026.

[46] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pp. 8634–8652, 2023. doi: 10.52202/075280-0377.

[47] Ruiyang Si, Jianxin Bi, Shunyu Yang, Rui Ni, Wenbo Huang, Qiang Wang, Shulong Jiang, Yintianrun Zhang, Feixiang Ruan, Kewei Zhang, Duomin Wang, Xiuyu Li, Haiwen Feng, Zhen Dong, and Daquan Zhou. Fewer tokens, better action: GPT-6 Astra robot agents with 14% higher success rate but 65% fewer tokens. *arXiv preprint arXiv:2610.01939*, 2026.

[48] Junyi Tang, Jie Peng, Zezhen Ding, Yuan Shen, and Tianlong Chen. AdaHVLA: Adaptive harnesses for long-horizon vision-language-action execution. *arXiv preprint arXiv:2609.29204*, 2026.

[49] Wenqing Tian, Zeyu Zhang, Zhaocheng Liu, Fengwei Liu, Qiang Liu, and Liang Wang. PhysEvo: Astra can act, let it. *arXiv preprint arXiv:2610.08995*, 2026.

[50] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL https://openreview.net/forum?id=ehfRiF0R3a.

[51] Kailin Wang, Haoxiang Jie, Yaoyuan Yan, Zhiyou Heng, and Zhaosong Li. RegenHarness: A robot agent harness with evidence-gated recursive self-improvement. *arXiv preprint arXiv:2609.27612*, 2026.

[52] Peidong Wang, Zhiming Ma, Ying Chang, Xufang Luo, Yiqun Zhang, Zihan Wang, Xiaocui Yang, Shi Feng, Yuqing Yang, and Dongsheng Li. Self-evolving embodied agents via skill-harness evolution. *arXiv preprint arXiv:2608.11350*, 2026.

[53] Xin Wang, Wenhao Wu, Menghao Zhang, Zhi Wang, Kun Shao, Jian Luan, Yang Li, Qing Li, Shangding Gu, Huichi Zhou, Shuqing Shi, Fei Ni, Shuo Lu, Weicheng Meng, Kang Li, Jin Wu, Kang Zhao, Shangmin Guo, Gen Li, Yongqiang Tang, Zhizhong Zhang, Yuan Xie, and Heng Qu. HarnessPAI: An evolving harness for physical AI. *arXiv preprint arXiv:2609.29166*, 2026.

[54] Yike Wang, Huaisheng Zhu, Zhengyu Hu, Yige Yuan, Zhengyu Chen, Shakti Senthil, Hannaneh Hajishirzi, Yulia Tsvetkov, Noah A. Smith, Pradeep Dasigi, and Teng Xiao. Rethinking the evaluation of harness evolution for agents. *arXiv preprint arXiv:2607.12227*, 2026.

[55] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An open, modular exploration towards systematic world-action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026.

[56] Wenli Xiao, Jia Xie, Tonghe Zhang, Haotian Lin, Letian "Max" Fu, Haoru Xue, Jalen Lu, Yi Yang, Cunxi Dai, Zi Wang, Jimmy Wu, Guanzhi Wang, S. Shankar Sastry, Ken Goldberg, Linxi "Jim" Fan, Yuke Zhu, and Guanya Shi. ENPIRE: Agentic robot policy self-improvement in the real world. *arXiv preprint arXiv:2606.19980*, 2026. arXiv comment: 2026 Conference on Robot Learning.

[57] Bingxin Xu, Yuzhang Shang, and Emilio Ferrara. Don't drop the BATON: Long-horizon robot manipulation via agentic subtask exploration and transition-aware memory. *arXiv preprint arXiv:2608.16889*, 2026.

[58] Lihe Yang, Bingyi Kang, Zilong Huang, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything: Unleashing the power of large-scale unlabeled data. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10371–10381, 2024.

[59] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything V2. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[60] Yifei Yao, Yude Zou, Guanbo Jiao, Xing Gao, Ning Gao, Yukai Wang, Jiantong Chen, Zihou Zhu, Xinyu Li, Ningyu Yan, Shuai Wang, Jianming Ma, Haoyu Guo, Weinan Zhang, and Chunhua Shen. Profiling a frontier agent with EBench: From direct control to harness with policy. *arXiv preprint arXiv:2610.35832*, 2026.

[61] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, Ayaan Malik, Kyungmin Lee, William Liang, Nadun Ranawaka, Jiasheng Gu, Yinzhen Xu, Guanzhi Wang, Fengyuan Hu, Avnish Narayan, Johan Bjorck, Jing Wang, Gwanghyun Kim, Dantong Niu, Ruijie Zheng, Yuqi Xie, Jimmy Wu, Qi Wang, Ryan Julian, Danfei Xu, Yilun Du, Yevgen Chebotar, Scott Reed, Jan Kautz, Yuke Zhu, Linxi "Jim" Fan, and Joel Jang. World Action Models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.

[62] Haoqi Yuan, Zhixuan Liang, Anzhe Chen, Ye Wang, Haoyang Li, Pei Lin, Yiyang Huang, Zixing Lei, Tong Zhang, Jiazhao Zhang, Jie Zhang, Jingyang Fan, Gengze Zhou, Qihang Peng, Chenxu Lv, Xiaoyue Chen, An Yang, Fei Huang, Junyang Lin, Dayiheng Liu, Jingren Zhou, Chenfei Wu, and Xiong-Hui Chen. Qwen-RobotManip technical report: Alignment unlocks scale for robotic manipulation foundation models. *arXiv preprint arXiv:2606.17846*, 2026.

[63] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026.

[64] Andy Zhai, Brae Liu, Bruno Fang, Chalse Cai, Ellie Ma, Ethan Yin, Hao Wang, Hugo Zhou, James Wang, Lights Shi, Lucy Liang, Make Wang, Qian Wang, Roy Gan, Ryan Yu, Shalfun Li, Starrick Liu, Sylas Chen, Vincent Chen, and Zach Xu. Igniting VLMs toward the embodied space. *arXiv preprint arXiv:2509.11766*, 2025.

[65] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations (ICLR)*, 2026.

[66] Wenbo Zhang, Kaixuan Wang, Yutao Ouyang, Xiaoyu Huang, Liyang Li, Kailun Su, Weiyang Jin, Wenhao Chai, Haotian Liang, Zhiyang Dou, Yue Chen, and Tianxing Chen. An unexpected robot policy: Early evaluations of GPT-6 Astra on RoboDojo and beyond. *arXiv preprint arXiv:2609.24170*, 2026.

[67] Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Yi Nie, Chunyang Zhu, Jiaxing Qiu, Yuchen Yan, Jiyuan Liu, Wenhao Tang, Jiaji Rao, Zhengru Fang, Changxu Wei, Yu Wang, Wenbo Ding, and Chao Yu. Harness VLA: Steering frozen VLAs into reliable manipulation primitives via memory-guided agents. *arXiv preprint arXiv:2607.08448*, 2026. Code: RLinf/RPent.

[68] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[69] Congjie Zheng, Chuanyi Xue, Bin Liang, Jun Yang, and Changshui Zhang. SEAGym: An evaluation environment for self-evolving LLM agents. *arXiv preprint arXiv:2606.17546*, 2026.

[70] Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, et al. X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. *arXiv preprint arXiv:2510.10274*, 2025. URL https://arxiv.org/abs/2510.10274.

[71] Hai Zhou, Fan Yang, Zeyi Chen, Lejun Zhang, Chenyang Fu, Jianping Li, Jiayi Zhu, Xianyang Liu, Yingxuan Yang, Lili Meng, Lu Wang, Xinyang Han, Jiankai Sun, Weirui Ye, Guanchu Wang, Wanhua Li, Ming Zhou, Boyi Li, Ming Jin, Kun Shao, Muhao Chen, Guang Chen, Lei Ma, Joey Tianyi Zhou, Costas Spanos, Xia Hu, Mac Schwager, Weinan Zhang, Qian Feng, and Shangding Gu. Who improves the robot? toward physical recursive self-improvement. *arXiv preprint arXiv:2610.18842*, 2026.

[72] Yude Zou, Yifei Yao, Guanbo Jiao, Jianming Ma, Guanjie Zheng, and Xing Gao. PICA-WAM: Visual perturbation invariance via cross-appearance self-alignment in world action models. *arXiv preprint arXiv:2610.26813*, 2026. URL https://arxiv.org/abs/2610.26813.

APPENDIX

A Information Protocol and Agent Interfaces **19**
A.1 Interface on EBench . . . 19
A.2 Task Memory Annotation . . . 20
A.3 Working Memory Tools and Reflection . . . 20
A.4 Interface on RoboDojo . . . 21

B Memory Examples **21**
B.1 Task Memory . . . 21
B.2 Working Memory . . . 22
B.3 Chain Dynamics . . . 23

C Per-Task Results on EBench **25**

D Per-Task Results on RoboDojo **26**
D.1 Comparison with Other Agentic Systems . . . 28

E Extended Ablations **28**
E.1 Memory Designs in Both Agent Harnesses . . . 28
E.2 Outcome Feedback . . . 29
E.3 Executed Trajectory of $\pi_{0.5}$ . . . 30
E.4 Comparison with the Critic-Driven RSI Loop . . . 30
E.5 Matched Controls per Task . . . 31
E.6 Coordinate Replay and Its Threshold . . . 32
E.7 Failure Modes of the Working Memory . . . 32

F Transfer Protocol **33**

G Cost and Wall-Clock Time **34**

H Case Studies **35**

A Information Protocol and Agent Interfaces

This section summarizes in Table A.1 the inputs and actions of each configuration of GPT-6-Astra under the protocol of Section 3.1 and reproduces the instructions and tool descriptions of the agent harness verbatim. The configurations differ in the tools exposed to the agent and in the memory retained across episodes.

Table A.1: Inputs and actions of the configurations of GPT-6-Astra on EBench. Proprioception covers the end-effector poses, the grippers, and the mobile base. The demonstration is one training recording per task, read through its annotation and the read-only demonstration tools. Outcomes are the terminal success and score of earlier adaptation episodes, and only SimplEvo receives them.

Input or action	Direct control	Harness with policy	SimplEvo
RGB views (overview, two wrists)	✓	✓	✓
Proprioception	✓	✓	✓
Instruction, robot constants	✓	✓	✓
Demonstration, annotation	✓	✓	✓
End-effector waypoints	✓	✓	✓
Calls to a post-trained policy	–	✓	–
Working memory	–	–	✓
Outcomes of earlier episodes	–	–	adaptation

A.1 Interface on EBench

Every EBench configuration of GPT-6-Astra receives the same base system instructions. Direct control appends the last sentence of the following prompt, *Choose continuous poses, motion timing and observation cadence yourself*, and the harnesses with policy append the description of their policy tool in its place.

```
You control one simulated EBench robot episode. Use only the supplied RGB and measured
robot state and the declared robot tools. Do not read external files, use shell, browse, or
delegate to other agents. The server judges success independently. Stop when it reports a
terminal result or a budget is exhausted. Repeated observe at the same timestep shows the
same scene. A closed gripper or finger contact alone does not prove a secure grasp. Choose
continuous poses, motion timing and observation cadence yourself.
```

The first message of an episode states the instruction, the robot specifications, and the resource limits, followed by guidance on grasp verification that all configurations share and by the paragraphs on the demonstration and the working memory. The message below opens a test episode of peg insertion with the fixed working memory.

```
Complete this single task yourself using the tools: Lift the peg and the hole at the same
time, put the peg into the hole, and then set them back on the table.

Robot: [...] two arms and a movable planar base. Left arm is left in the overlook view.
EEF poses are local to each arm's planner base: +X forward, +Y robot-left, +Z up;
quaternion order wxyz. Fingers extend along tool +X and open along tool Y. Wrist cameras
move/rotate with the hands. Measured local EEF z is not height above the tabletop. There
is no supplied depth, object pose or camera calibration. Only RGB and measured robot
EEF/gripper/base state are available. Base observed yaw is radians; tool command yaw is
degrees. You have one continuous conversation and may choose any continuous EEF target,
both arms, base motion and waypoint batching. Use execute_eef sequentially, not parallel
calls. Use observations to judge motion and object manipulation. The server determines
task success independently. Do not start another episode. Resource limits: unlimited tool
calls, physics deadline enforced by the server; no hard token limit; [...] Select only
useful camera views after the initial observation to control resource use. When finished,
state the observed outcome concisely.

Manipulation guidance: after visually aligning, close at a stationary pose and allow
settling. A nonzero finger gap on the table is contact evidence, not proof of a secure
grasp. Test each grasp by lifting only 0.01 m first, using motion_steps=16 or more, then
hold 8..16 steps and request the active wrist plus overlook views. Keep orientation and
base fixed during this test. Check both finger gap and RGB to confirm the object rises with
the gripper before further lifting or transport. If it remains on the table or slips, lower
and adjust local contact depth/centering using current images. There is no fixed limit on
failed grasp attempts; retry or change approach within the remaining experiment budgets.
Avoid large instantaneous target jumps while holding an object; use motion_steps for
```

```
controlled transport.  Total steps per waypoint must remain <=64 and includes the ramp.  Use
only the active wrist for local corrections, adding overlook when establishing alignment,
testing grasp, or verifying placement.

RGB inputs are downsampled:  overlook 640x360, wrist 384x288; no crop or object annotation.

The saved ICL demonstration remains available through query_icl throughout this episode.
Revisit it when the target region, stage order or arm roles are unclear, and when repeated
failures or a change of approach make the reference useful.  Use view=text for the original
annotation and keyframe index, view=frames with frame_indices for selected images, or
view=all for the full reference.  Indices are zero-based entries in the keyframe index, not
source video frame numbers.  Querying does not advance simulation.  Ground actions in the
current RGB/state.

Working memory:  read_task_notes holds notes about this task recorded by earlier attempts at
it in other scenes (it may be empty); they are read-only in this attempt.
read_working/write_working is this attempt's scratchpad; history, inspect and
read_state_history give exact records and archived views of this attempt; each tool result
states its call number.  Before each robot action, briefly predict what you expect to see;
after it, describe what actually changed.  Notes are hypotheses:  verify them against the
current views.
```

The agent commands the robot through three tools. The tool `execute_eef`, whose description follows, executes all arm and base actions. The tool `set_gripper` commands a normalized gripper opening while both arms and the base remain stationary, and `observe` returns the current camera views and robot state without advancing the simulation.

```
Execute 1..6 native GenManip absolute EEF waypoints in order.  Each arm target has xyz(m),
unit quaternion(wxyz) in that arm's planner-base frame, and two finger joint targets(m):
[0.044,0.044] open, [0,0] close.  No translation or angle quantization.  Both arms and the
mobile base may move together.  Null arm holds its measured local pose and prior gripper
target; null base holds measured planar base pose.  Base x/y are metres; command yaw is
DEGREES, while observed state.base yaw is RADIANS. Each waypoint consumes 'steps' physics
steps (1..64).  Optional motion_steps (0..steps, default 0) linearly ramps position and
spherically interpolates rotation from measured pose to target over that many physics steps,
then holds for the remainder.  Base pose also ramps.  Finger commands apply immediately, so
close in a separate stationary waypoint.  With motion_steps=0 the full target applies
immediately:  extra steps are holding time, not slow motion.  All commands are absolute;
there is no repeated relative displacement.  The simulator performs IK; targets may remain
unreachable, so inspect returned measured poses.  No automatic grasp approval or task-stage
constraints are imposed.  Returns final robot state and selected RGB cameras; sequences have
no intermediate model observation.  You choose target poses, timing, observation cadence,
camera subset and when to batch waypoints.
```

A.2 Task Memory Annotation

The annotator is an instance of GPT-6-Astra at high reasoning effort with three read-only tools. The tool `demo_search` divides the recording into action segments and gripper events and reports, for each segment, the moving arm or base, its displacement, and the measured gripper opening. The tool `demo_inspect` returns frames from the overview and wrist cameras, and `demo_actions` reports the measured arm and base motion over a frame range relative to its first frame. The annotator receives the following message, here for peg insertion, and submits the task memory once.

```
You are annotating one recorded robot demonstration so that a robot agent can learn from it
later.  The robot is a mobile bimanual manipulator; the demonstration was recorded in
simulation.  Task instruction:  "Pick up the peg and the hole simultaneously, insert the peg
into the hole, and place them back on the table.".  The demonstration has 1943 frames (one
frame is one execution step).

Use demo_search to list its motion bouts and gripper events, demo_inspect to look at frames
from the overlook, left_wrist and right_wrist cameras, and demo_actions to read measured arm
motion.  Review the WHOLE demonstration, including its last frames:  stages are easy to miss
when a scene is dark or an arm moves without a gripper event (for example closing a door or
a lid).  Then call submit_icl once.

Write for a robot agent that will face a different layout of the same task:  describe
operations, arm roles, order, contacts and the visible end state; say what you saw versus
what you infer; do not report absolute coordinates or frame-specific numbers as
instructions.  There is no success label for this recording; the instruction defines what
must be true at the end.  [...]
```

A.3 Working Memory Tools and Reflection

During an episode, the MLLM reads the working memory through `read_task_notes`. It also keeps a scratchpad for the current episode (`read_working` and `write_working`), lists its tool calls (`history`), revisits frames it received (`inspect`), and reads the measured robot

states of the episode (`read_state_history`). It can also revisit the demonstration through `query_icl`, which returns the annotation and selected keyframes, and through the three demonstration tools of the annotator. During adaptation, the MLLM rewrites the working memory through `write_task_notes`, whose description follows.

```
Replace your task notes (at most 32768 bytes UTF-8; nothing is truncated).  [...]  Write
reusable, revisable, task-specific knowledge:  what worked, what failed and why, with the
evidence (attempt and call numbers).  Coordinates measured in one scene are not commands for
another scene.
```

After each adaptation episode, the agent receives one reflection turn in which only the memory tools are available. The turn opens with the following message, and the variant without outcome feedback of Appendix E.2 replaces the result with a statement that the official result is not shown.

```
The episode has ended.  Official result from the evaluation server:  success=<SR>,
score=<Score>.  No robot action is possible now; only the memory tools work.  Review this
attempt if useful, then update your task notes with write_task_notes (at most 2 writes) so
that later attempts at this task in other scenes do better:  what worked, what failed and
the evidence (attempt and call numbers).  Keep the notes reusable and revisable.  Reply with
one short sentence when done.
```

A.4 Interface on RoboDojo

The RoboDojo agent harness retains the tools and the message structure of EBench, with the robot paragraph adapted to the fixed bimanual tabletop setup.

```
Robot:  two ARX X5 6-DoF arms with parallel grippers on a fixed table (no mobile base).
Shared world frame (metres):  the robot faces +Y; the left arm's base is at (-0.3, -0.45,
0.765) and the right arm's at (0.3, -0.45, 0.765), so +X is toward the robot's right and +Z
is up; the tabletop is at z = 0.765.  EEF poses are the arm flange (link6) pose in this
frame, quaternion order wxyz.  The fingers extend along the flange's +X axis and the jaws
open and close along its Y axis; the centre of the finger pads is about 0.15 m from the
flange origin along +X. For example, quaternion (0.5, -0.5, 0.5, 0.5) points the fingers
straight down with the jaws closing along world X. Each episode starts with both arms at the
same home pose (all joints zero, grippers open).  Cameras:  cam_head (fixed above and behind
the arms, looking down at the table), cam_left_wrist and cam_right_wrist (move and rotate with
the hands).  There is no supplied depth, object pose or camera calibration.  Only RGB and
measured EEF/gripper state are available; the gripper value is your last command (1 open, 0
closed), not a measured finger width.
```

B Memory Examples

This section presents the two memories of SimplEvo introduced in Sections 3.2 and 3.3 as the agent reads them and traces how the working memory develops along the adaptation chains. All excerpts are quoted verbatim, and [. . .] marks omitted text.

B.1 Task Memory

Figure B.1 reproduces the task memory of peg insertion on EBench. It describes the arm roles and 11 stages in order and closes with the observed outcome. Like the task memories of the other EBench tasks and the 34 demonstration-equipped tasks of RoboDojo, it contains no coordinates. In every episode of the task, the MLLM reads this text together with 15 keyframes of the recording, each with a one-line caption.

One recorded demonstration of this task. Reference instruction: Pick up the peg and the hole simultaneously, insert the peg into the hole, and place them back on the table.
Arm roles: The left arm grasps the outside of the receiving hole block, lifts and positions it with its opening accessible, holds it nearly stationary during insertion, and finally places the assembled block and peg on the table. The right arm grasps the yellow peg, extracts it from a separate tabletop holder, aligns and inserts it into the left-held block, releases it, and withdraws. The base remains essentially stationary. The observed pickups are sequential, with the hole picked up first; both objects are held concurrently during alignment and insertion.
Observed sequence: Initially a receiving block with an empty central hole and a separate holder containing an upright yellow peg rest on the table. The left gripper approaches, opens around the receiving block, closes against its outer sides, and lifts and shifts the block toward the shared workspace. While the left arm holds the hole accessible, the right gripper approaches and closes around the exposed peg. The right arm lifts the peg out, leaving its original holder on the table. It carries the peg toward the left-held opening and adjusts its orientation and lateral position to bring the lower end into alignment. With the left block steady, the right arm gradually advances the peg into the central hole, making small pose corrections. The right gripper opens, leaving the peg protruding from and supported by the receiving block, then lifts away and retracts. The left arm lowers the assembled pair onto the tabletop, opens its gripper, lifts clear, and returns to rest. The final view shows the peg still in the receiving block, the original holder empty nearby, and both arms withdrawn.
Stages (frame = one execution step of the recording):
- frames 0-25 (none; seen): Initial tabletop arrangement. Both hands begin clear of the objects. An empty receiving block and a separate holder containing the yellow peg sit on the tabletop.
- frames 25-192 (left; seen): Approach and surround the receiving block. The left wrist turns and approaches the receiving block, opens its fingers, and lowers them to opposite outer sides while leaving the central hole unobstructed.
- frames 192-354 (left; seen): Grasp, lift, and hold the receiving block. The left fingers close on the block; the arm lifts and shifts it toward the assembly workspace, then holds the opening accessible. The right hand remains clear.
- frames 354-562 (right; seen): Approach the peg while holding the hole. The right arm approaches the yellow peg, opens, and lowers its fingers around the exposed shaft. The left arm maintains its grasp on the receiving block.
- frames 562-832 (right; seen): Grasp and extract the peg. The right fingers close around the peg and hold it while the arm gradually lifts it out of its original holder. A clear gap appears between the peg tip and the now-empty holder. The left hand continues holding the receiving block.
- frames 832-1100 (right; seen): Carry and align the peg over the receiving hole. The right arm brings the free peg toward the left-held block, changing wrist orientation and lateral position so its lower end approaches the central opening. The left arm remains nearly stationary.
- frames 1100-1479 (right; seen): Advance the peg into the held hole. The right arm lowers and adjusts the peg at the opening while keeping its grasp. The shaft becomes visibly engaged in the receiving hole. Small alignment and insertion corrections continue with the left gripper supporting the block; the exact contact onset and seating depth are partly occluded.
- frames 1479-1679 (right; seen): Release the inserted peg and withdraw the right hand. The right fingers open, leaving the peg standing in the receiving block. The right hand rises clear, closes empty during retraction, and returns to its resting region. The left hand holds the assembled pair.
- frames 1679-1822 (left; seen): Lower the assembled pair to the table. The left arm lowers the receiving block with the peg still inserted, placing its base against the tabletop and holding it briefly before release.
- frames 1822-1855 (left; seen): Release the assembly and lift clear. The left fingers open away from the block, and the hand lifts clear. The peg and receiving block remain together on the table.
- frames 1855-1943 (left; seen): Retract the left arm and leave the final arrangement. The left gripper closes empty as the arm retracts to rest. Through the last frame, both hands remain clear, the peg stays inserted in the receiving block on the table, and the original empty holder remains nearby.
Observed outcome: The yellow peg is visibly inserted in the receiving block and remains there after both grippers release. The receiving block and peg rest together on the table at the end; the separate original peg holder remains empty on the table. Both arms are clear and their empty grippers are closed. This supports the insertion-and-return end condition. [. . .]
This is a separate recorded episode with no success label; the live instruction defines what must be true at the end. Transfer operations, order and checks; ground every command in the live views and measured state.

Figure B.1: Task memory of peg insertion on EBench, written once by the annotator from one training recording and read in every training and test episode of the task.

B.2 Working Memory

Figure B.2 shows excerpts of two fixed working memories. On EBench peg insertion, a chain of nine training episodes with five successes wrote the memory that all 20 test episodes read, and one of its lessons warns that a single camera view cannot confirm that the peg is aligned with the hole. On RoboDojo stack blocks, all 15 training episodes succeeded, and one lesson prefers stacking on an existing reachable base over moving the base to a central site.

(a) EBench, peg insertion

Task: pick up receiving block and peg, insert while both held, return assembly to table. Attempt 2 official success=1.0, score=1.0; terminal at call 44, timestep ∼1311 during lowering assembled pair. Attempt 1 failed at 2500 steps (score .5). Preserve physics for placement; unlimited calls are not unlimited physics. Coordinates below are scene-specific evidence, not commands to replay.
[. . .]
- Left calls 4-7: open approach above receiving block; lower around outer sides; stationary close 24 steps; 1 cm test lift with 20 ramp+12 hold. Pose at grasp [.2807,-.0402,.0468], q [.8398,.1282,.2068,-.4854]. Finger joints ∼.014 each. Block stayed fixed in wrist image and visibly lifted in overlook.
[. . .]
- Crucial: single-view tip overlap above hole is not 3D coaxial alignment. Peg/hand may maintain apparent position until rim contact, then slide/tip. At first stationary tip, changed cap attitude, or changing finger gap, relieve pressure promptly; do NOT add 1 cm or more descent. Use 1-3 mm corrections near contact.
[. . .]
- Call16 applied Exp([22.52,-20.14,35.96]deg) to current right pickup quaternion, yielding q [.82513055,-.04968560,.12816274,.54796854]. Moved right [.310,.156,.185],48ramp+8hold. Achieved q verified. This turn remained FIXED for all subsequent successful insertion commands.
[. . .]
- Calls25-28: FIX XY [.308,.138] and q; descend z .121,.117,.113,.109 in 4mm increments, each16ramp+8hold, both wrists.
[. . .]
2. Single left-view tip overlap can occur while the peg is still centimeters ABOVE the opening. Right wrist was useful after wrist reorientation: track tip-to-hole separation in BOTH views during descent.
[. . .]
Updated reusable inference: Near-identical pickup/turn EEF poses can yield a different peg-in-hand pose, as attempts6/8/9 demonstrate. Stable gap∼.00360 can be successful (attempt9), whereas a contact-induced decrease from an established larger gap warns of sliding. Absolute gap value alone is not the criterion. Use both wrist views, stable relative motion, and small live XY corrections; gradual loosening after promising engagement can reveal support.

(b) RoboDojo, stack blocks

GENERAL GUIDANCE: Attempts1-15 officially succeeded. Prefer two transfers onto an existing reachable base; three-transfer central relocation is robust but often takes ∼500steps. Verify reach, size, and alignment from current RGB, not historical coordinates. Standard down q=[.5,-.5,.5,.5], typical pickup flange z=.9255.
[. . .]
Keep orientation fixed while holding. Stationary close10 then mandatory1cm lift6ramp+4hold, head+wrist: confirm object moves with fingers, not just finger contact/command.
[. . .]
WRIST: image right=world+X; moving hand-worldY moves object upward. Supporting cube appears ABOVE held cube when aligned, not overlapping image centers.
[. . .]
Prefer a central base from outset when blocks span both sides; this would save the failed transport and recovery time. Reachability boundary observed here is pose-specific, not a universal X limit.
[. . .]
Base released at flange .928, middle .963, top .998; all used [0,-.21].
[. . .]
Keep tilt FIXED throughout transfer so the cube stays upright; rotating gripper back to downward while holding would rotate the cube. Call18 raised/retracted to[-.08,-.12,1.06] (18 ramp+2 hold), then moved to[-.26,-.062,1.07] (22+2). Offset flange Y to account for forward fingers, instead of reusing downward-grasp flange XY.
[. . .]
LOWERING solved high reach limitation; no need to repeat unreachable height or regrasp. Here .07m clearance left fingertips visibly above tower before retreat. Retain geometry/RGB checks, not coordinates.

Figure B.2: Excerpts of two fixed working memories of GPT-6-Astra: (a) peg insertion on EBench and (b) stack blocks on RoboDojo. Blue marks sentences that contain a stored pose (a position in meters or an orientation quaternion), and black marks the remaining sentences.

B.3 Chain Dynamics

On EBench, the 26 adaptation chains comprise 192 training episodes with 101 successes. Of these chains, 21 stop after three consecutive successes, 11 of them without a failure, and the other five reach the limit of 15 episodes. A training episode consumes 4.09M tokens on average, including its reflection turn.

Table B.1: Online RoboDojo chains by blocks of five positions (210 episodes per cell): score (0–100) and mean tokens per episode for task memory only and for SimplEvo online, and the mean size of the working memory read (KB). Scores average episodes without dimension weighting, and Figure B.3a gives the SR.

	Task memory only		SimplEvo online		
Positions	Score	Tokens	Score	Tokens	Memory
1–5	41.55	1.42M	55.93	1.65M	5.4
6–10	39.64	1.42M	62.88	1.75M	19.7
11–15	42.76	1.41M	67.60	1.77M	25.6

On RoboDojo, averaging Figure B.3a over blocks of five positions, the online SR rises from 48.57% in the first block to 61.43% in the last, while task memory alone stays between 32.38% and 36.19%. Table B.1 gives the score, the token count per episode, and the memory size of each block. The

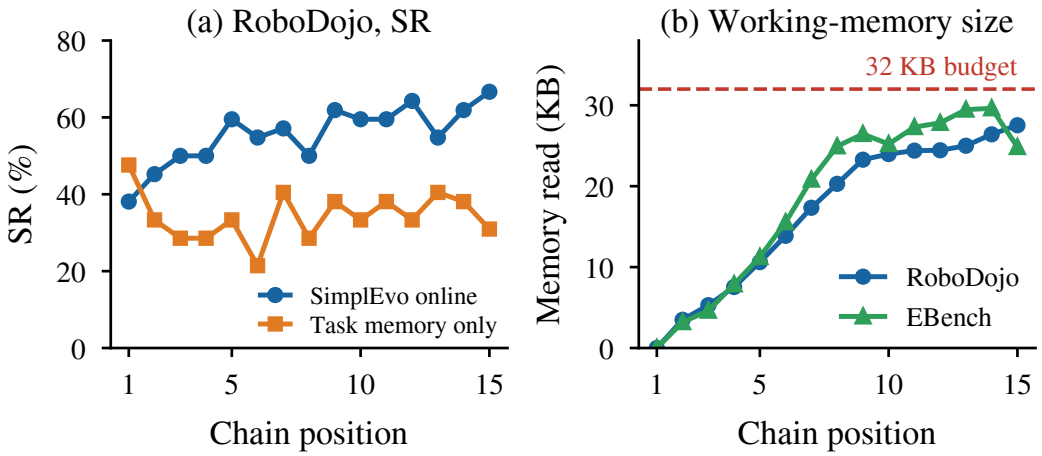

Figure B.3: Adaptation chains of SimplEvo. (a) SR at each chain position of the online RoboDojo chains (42 episodes per point, averaged over tasks without dimension weighting) with working memory and with task memory only. (b) Mean size of the working memory read at each position, over the 42 RoboDojo chains and over the EBench adaptation chains still running at that position (26 at positions 1 to 3, 5 at positions 14 and 15).

working memory grows by about 3 KB per episode over the first nine positions and then levels off below the budget, and the reflection turn adds 0.21M tokens per episode on average.

C PER-TASK RESULTS ON EBENCH

Table C.1: Per-task results on the 510 test episodes of EBench. Precision is low (L), medium (M), or high (H), and the horizon is short (S) or long (L). Train gives the successes and episodes of the adaptation chain, which stops after three consecutive successes or 15 episodes and yields the fixed working memory. The SR columns of GPT-6-Astra give the configurations without working memory in Table 1 under direct control and in the harnesses with $\pi_{0.5}$ and InternW0-Δ (IW0-Δ). Replay is the mean percentage of moving waypoints in Appendix E.6 that come within 1 cm of a coordinate in the fixed working memory, and a dash marks a memory that stores no coordinates. Seen / unseen gives the successes of SimplEvo on test episodes whose layout appears, or does not appear, among the training episodes of the chain, and a dash marks single-layout tasks or a split without test episodes. The last row gives totals for Train and n and means over tasks in the other columns. Bold marks the best SR in each row.

						SR (%), GPT-6-Astra			SimplEvo			
Task	Prec.	Hor.	Layout	Train	n	Direct	$\pi_{0.5}$	IW0-Δ	SR	Score	Replay	Seen / unseen
apple from shelf	L	S	multi	3/3	20	70.0	**85.0**	75.0	**85.0**	85.0	–	9/9 / 8/11
apple to fruit bowl	L	S	multi	5/6	20	90.0	95.0	**100.0**	**100.0**	100.0	7	8/8 / 12/12
bookmark on book	L	S	multi	3/3	20	**90.0**	85.0	**90.0**	**90.0**	90.0	34	3/3 / 15/17
bottle	M	L	multi	3/13	20	0.0	0.0	0.0	**45.0**	88.8	27	5/10 / 4/10
bowl to plate	L	S	multi	3/3	20	75.0	**95.0**	**95.0**	80.0	80.0	19	3/5 / 13/15
collect coffee beans	H	L	single	5/13	20	0.0	0.0	0.0	**90.0**	95.0	89	–
detergent	L	L	multi	3/3	20	55.0	**90.0**	70.0	**90.0**	96.7	16	1/2 / 17/18
dish	M	L	multi	4/5	20	50.0	60.0	70.0	**90.0**	98.0	–	4/4 / 14/16
dishwasher	L	L	multi	3/3	20	5.0	60.0	75.0	**85.0**	94.2	26	– / 17/20
flip cup collect cookies	M	L	single	4/15	20	10.0	40.0	**60.0**	45.0	80.0	39	–
frame against pen holder	L	L	single	3/3	20	65.0	65.0	45.0	**100.0**	100.0	57	–
fruit	L	L	multi	3/3	20	40.0	30.0	45.0	**95.0**	98.0	3	– / 19/20
install gear	H	S	single	3/15	20	15.0	10.0	40.0	**85.0**	87.5	76	–
make sandwich	M	L	multi	4/10	15	46.7	73.3	**86.7**	73.3	93.3	35	8/12 / 3/3
microwave	L	L	multi	5/6	15	73.3	60.0	60.0	**86.7**	97.8	71	3/3 / 10/12
peg in hole	H	L	single	5/9	20	20.0	5.0	0.0	**85.0**	92.5	69	–
pen	M	L	multi	7/15	20	15.0	5.0	0.0	**25.0**	70.0	1	5/20 / –
perfume to cosmetics rack	L	S	multi	3/5	20	75.0	80.0	80.0	**95.0**	97.5	29	4/5 / 15/15
put glass in glassbox	M	L	single	3/6	20	20.0	**85.0**	80.0	70.0	78.3	63	–
remote to holder	L	S	multi	3/3	20	**100.0**	90.0	**100.0**	**100.0**	100.0	5	4/4 / 16/16
salt to spice rack	L	S	multi	3/3	20	**100.0**	**100.0**	**100.0**	**100.0**	100.0	7	3/3 / 17/17
shop	L	L	multi	8/11	20	0.0	15.0	5.0	**95.0**	99.2	26	19/20 / –
soap to dish	M	S	multi	3/3	20	95.0	90.0	**100.0**	90.0	90.0	21	5/5 / 13/15
teacup to saucer teapot to tray	M	S	multi	3/3	20	85.0	30.0	55.0	**95.0**	97.5	7	5/5 / 14/15
tighten nut	H	L	single	4/15	20	10.0	15.0	10.0	**70.0**	85.0	70	–
utensils to holder	L	S	multi	5/15	20	10.0	15.0	15.0	**60.0**	77.5	8	4/10 / 8/10
All tasks				101/192	510	46.7	53.0	56.0	**81.7**	91.2	34	

Table C.2: SR (%) and score of GPT-6-Astra with SimplEvo on EBench by task group: the first episode of each of the 26 adaptation chains, which reads only the task memory, all 192 training episodes, and the 510 test episodes of Table 1 with the fixed working memory. SR and score are averaged within each task and then across tasks.

		Overall		Precision			Mobility		Horizon	
Episodes	n	SR	Score	Low	Med.	High	Mobile	Tabletop	Short	Long
First training episode (task memory only)	26	57.7	72.2	85.7	37.5	0.0	73.7	14.3	72.7	46.7
All training episodes (online)	192	70.8	83.6	88.1	58.3	35.2	80.1	45.3	81.5	62.9
Test episodes (fixed working memory)	510	81.7	91.2	90.1	66.7	82.5	83.2	77.9	89.1	76.3

The seen/unseen partition groups existing test episodes by layout overlap with completed adaptation episodes. Tables C.1 and C.2 break down the EBench results of Section 4.2 by task and over the adaptation chains, and Table 6 in Section 5.3 compares seen and unseen layouts. Across the 26 tasks, the fixed working memory improves or matches the SR of direct control on 25 tasks and improves or matches the stronger of the two harnesses with policy on 21 tasks.

Coordinate replay, a moving waypoint of Appendix E.6 within 1 cm of a coordinate in the fixed working memory, concentrates on the seven single-layout tasks. Over their 140 test episodes, a mean of 66.1% of the moving waypoints lies within 1 cm of a stored coordinate, against 19.2% over the 330 episodes of the 17 multi-layout tasks whose working memory stores coordinates. Scored

against the same working memories, the direct-control episodes on the same test scenes reach 4.3% and 1.0%, which gives the share expected without working memory. On the other two multi-layout tasks, apple from shelf and dish, the fixed working memory stores no coordinates and reaches 85.0% and 90.0% SR.

Adaptation chains. Table C.2 shows that in every task group, the SR rises from the first training episode to all training episodes and again to the test episodes. GPT-6-Astra succeeds in the first chain episode on 15 of the 26 tasks but on no high-precision task. The rise from training to test is largest on the high-precision tasks, where the chains run 9 to 15 episodes and the reflection turns mostly analyze failed attempts.

D PER-TASK RESULTS ON ROBODOJO

Table D.1 breaks down Tables 2 and 3 by task, and the remainder of this section compares SimplEvo with other agentic systems. On the test layouts, the fixed working memory improves or matches the official zero-shot entry of GPT-6-Astra on 41 of the 42 tasks and reaches at least 90% SR on 18 tasks. On 9 of the 12 Generalization tasks, randomization costs SimplEvo at most four SR points or raises its SR. On the 15 training layouts, online adaptation improves or matches the SR of task memory alone on 39 of the 42 tasks.

Table D.1: Per-task SR (%) of GPT-6-Astra on the 42 tasks of RoboDojo, grouped by capability dimension. The official zero-shot entry covers 50 layouts per task. Task memory only and SimplEvo online run on the same 15 training layouts per task, and the fixed working memory is evaluated on the 50 test layouts per task. On Generalization tasks, each value averages the standard and randomized configurations. The overall SR averages the five dimensions, and bold marks the best value in each row.

	Official	Online, 15 training		Fixed
Task	zero-shot	Task mem.	SimplEvo	50 test
Generalization				
arrange largest number	60.0	79.5	67.0	**92.0**
fold clothes	72.0	72.3	80.4	**90.0**
hang mugs	0.0	0.0	**6.2**	2.0
make toast	0.0	0.0	**20.5**	16.0
pack objects into box	0.0	0.0	7.1	**12.0**
pour liquid into cup	4.0	12.5	46.4	**70.0**
push T	**60.0**	26.8	21.4	34.0
sort nesting dolls by size	18.0	12.5	32.1	**60.0**
stack blocks	86.0	79.5	**100.0**	98.0
stack bowls	62.0	59.8	74.1	**88.0**
store laptop and headphones	**0.0**	**0.0**	**0.0**	**0.0**
sweep blocks	4.0	6.2	46.4	**62.0**
Precision				
build tower	2.0	53.3	73.3	**94.0**
deposit coin	2.0	26.7	73.3	**92.0**
fasten screws	16.0	**33.3**	13.3	28.0
insert key	0.0	0.0	6.7	**16.0**
insert tubes	0.0	13.3	40.0	**60.0**
play xylophone	8.0	66.7	**100.0**	98.0
plug in charger	0.0	0.0	40.0	**80.0**
pour balls into vase	4.0	0.0	20.0	**48.0**
Long-Horizon				
classify objects	56.0	60.0	86.7	**96.0**
fill egg holder	0.0	0.0	6.7	**22.0**
fill pen holder	0.0	0.0	6.7	**14.0**
make kong	0.0	33.3	86.7	**96.0**
organize table	0.0	13.3	13.3	**28.0**
play stacking toy	0.0	0.0	6.7	**24.0**
play tic tac toe	0.0	93.3	**100.0**	**100.0**
put bottles into dustbin	10.0	26.7	80.0	**88.0**
Memory				
cover blocks	46.0	73.3	**100.0**	**100.0**
imitate sorting sequence	36.0	40.0	73.3	**94.0**
match and pick from conveyor	62.0	66.7	73.3	**88.0**
press by number	70.0	**100.0**	**100.0**	**100.0**
swap blocks	0.0	80.0	**100.0**	**100.0**
swap T	18.0	46.7	73.3	**92.0**
Open				
align blocks	50.0	60.0	**100.0**	**100.0**
classify objects by language	30.0	33.3	**86.7**	76.0
general pickup	**84.0**	66.7	73.3	**84.0**
pick from conveyor by image	4.0	13.3	60.0	**72.0**
pour by language	0.0	0.0	73.3	**98.0**
solve equation	40.0	40.0	80.0	**92.0**
stack blocks by language	40.0	46.7	86.7	**98.0**
store tools in toolbox	**0.0**	**0.0**	**0.0**	**0.0**
Overall (mean of dimensions)	22.48	36.37	58.53	**69.63**

D.1 Comparison with Other Agentic Systems

Table D.2: Score (0–100) on the seven RoboDojo tasks evaluated by RAX [23], with the first three tasks in their randomized configurations. Italic rows come from the RAX paper with at most 16 or 64 exploration episodes per task, and its programs run on metric depth and camera poses outside our information protocol. SimplEvo uses the working memory fixed after 15 training episodes and is evaluated on its test layouts.

System	Toast	Mugs	Sweep	Plug	Swap	Kong	Pour	Mean
RAX, ≤16	*2.5*	*36.9*	*53.3*	*94.0*	*100.0*	*64.0*	*100.0*	*64.4*
RAX, ≤64	*47.2*	*39.2*	*71.7*	*94.0*	*100.0*	*96.0*	*100.0*	*78.3*
SimplEvo, 15	26.0	24.0	56.0	80.0	100.0	96.0	98.0	68.6

Three agentic systems report RoboDojo results outside the public leaderboard. COAP [22] executes synthesized code policies on camera parameters and object sizes outside our information protocol and reports 70.24% SR. RAX [23], whose coding agent explores each task and submits a program that runs without the agent on metric depth and camera poses, evaluates seven of the 42 tasks and reports scores, which Table D.2 compares with SimplEvo. With at most 16 exploration episodes per task, RAX scores below SimplEvo, and with up to 64 it scores higher, mainly on the three randomized tasks and the plug task. SenseNova [44] provides GPT-6-Astra with a knowledge package fixed before testing and reports an unweighted task-averaged SR of 27.62% on five layouts per task under the official step limit at 12.47M tokens per episode. Under the same unweighted average over the 42 tasks on the test layouts, the fixed working memory of SimplEvo reaches 66.71% SR at 1.49M tokens per episode.

E Extended Ablations

This section details the protocols and per-task results of the ablations in Section 5. Appendix E.1 covers memory designs in both agent harnesses, Appendix E.2 outcome feedback, Appendix E.3 the executed trajectory of $\pi_{0.5}$, Appendix E.4 the comparison with the critic-driven RSI loop, Appendix E.5 matched controls across layouts, Appendix E.6 the distance threshold of coordinate replay, and Appendix E.7 how the working memory fails.

E.1 Memory Designs in Both Agent Harnesses

Protocol. Each design runs one chain of ten validation scenes per task in the same scene order, and the first episode of each chain reads the task memory alone. The designs differ only in the information retained across episodes. The first design reads only the task memory in every episode. The second design carries the text working memory of SimplEvo. The third design adds visual replay to the text working memory. The replay archives the frames and robot states of earlier attempts for inspection during execution and reflection. Both agent harnesses run these three designs on the four high-precision tasks. In the harness with $\pi_{0.5}$, a fourth design, described in Appendix E.3, adds the executed trajectory of every policy call to the text design on the four high-precision tasks. Under direct control, a continuous design adapted from VISTA [20] replaces the working memory with one conversation across the ten episodes of each chain, together with the task memory, a guide that the MLLM revises, and an archive of observed frames. Time counts the control phase of an episode, from the first observation until the MLLM ends control, as model time plus 0.5 s per physics step.

Results. Tables 4 and 5 list the results. In the harness with $\pi_{0.5}$, the clearest effect of memory concerns completion judgment on collecting coffee beans. With task memory only, the MLLM ends control before the evaluator reports a result in all ten episodes of this task, and with the text working memory and with the visual replay in five of ten. The successes of the task stay at 0 and 1, and its mean score rises from 0 to 31.4 and 27.9. Under direct control, on gear installation, peg insertion, and nut tightening, the agent without working memory already grasps near the poses that the final working memory records, so the working memory mainly changes the actions after the grasp. The MLLM rarely consults the visual replay in either agent harness.

Memory in the harness with $\pi_{0.5}$. Working memory changes how the MLLM uses the harness with $\pi_{0.5}$ but leaves its success nearly unchanged. Each policy call returns the executed step count, the robot state, and the camera images after the call, and the MLLM keeps the language instruction of the policy unchanged, so the trajectory of the policy stays outside the context of the MLLM. The reflection therefore records when to call the policy and for how many steps, and the same recipe can succeed in one scene and fail in the next. With working memory, the MLLM moves control from the policy to its own end-effector commands. The share of the commanded physics steps that $\pi_{0.5}$ executes falls from 53% over the first three episodes of each chain to 36% over the last four, while it stays between 55% and 60% with task memory only. These end-effector commands return to coordinates of the working memory about half as often as under direct control. The critic-driven RSI loop of Appendix E.4 follows the same pattern, as $\pi_{0.5}$ executes 72.7% of the commanded steps with the empty playbook and 67.5% after five rounds.

E.2 Outcome Feedback

Table E.1: SR (%) and mean score of GPT-6-Astra over the 20 episodes of each of eight chains, with the working memory written without and with the official success and score of each episode. Bold marks the better SR and score in each row.

		Without outcomes		With outcomes	
Task	**Layouts**	**SR**	**Score**	**SR**	**Score**
collect coffee beans	single	35.0	54.6	**50.0**	**71.4**
install gear	single	25.0	35.0	**65.0**	**65.0**
peg in hole	single	**90.0**	**95.0**	65.0	80.0
tighten nut	single	**60.0**	**80.0**	30.0	65.0
dishwasher	multi	**95.0**	**98.3**	90.0	**98.3**
utensils to holder	multi	45.0	67.5	**95.0**	**97.5**
fruit	multi	70.0	92.0	**95.0**	**98.0**
dish	multi	**100.0**	**100.0**	**100.0**	**100.0**
Mean, high-precision		**52.5**	66.2	**52.5**	**70.4**
Mean, multi-layout		77.5	89.5	**95.0**	**98.5**
Mean, 8 tasks		65.0	77.8	**73.8**	**84.4**

Protocol. To evaluate how withholding the official success and score affects the evolution of M_k^{work} from scratch, eight chains run online adaptation across 20 scenes of their task in a fixed order from an empty working memory, once with the official success and score after each episode and once without them, with the same task memory, prompt, tools, step limits, and size budget. The tasks are the four high-precision tasks, all single-layout, and the multi-layout tasks of dishwasher loading, utensil placement, the fruit task, and dish placement. Without outcome feedback, every episode ends with the same message, the record of earlier attempts omits the success, score, and step count of each attempt, and the reflection prompt states that the official result is not shown.

Results. In Table E.1, withholding outcomes lowers the mean score over the eight chains from 84.4 to 77.8. The direction varies across tasks, as the SR falls on four, rises on three, and ties on one. On the four high-precision tasks, the SR stays at 52.5% in both conditions, with peg insertion and nut tightening higher and gear installation and coffee-bean collection lower without outcomes. On the four multi-layout tasks, the SR falls from 95.0% to 77.5%, mostly through utensil placement and the fruit task. Over the last five positions of each of these chains, which the matched control of Appendix E.5 also covers, the mean score is 71.2 with task memory only, 90.0 without outcomes, and 98.0 with outcomes. An episode without outcomes takes 3.29M tokens on average, against 2.67M with outcomes.

E.3 Executed Trajectory of $\pi_{0.5}$

As defined in Section 3.1, a policy call reports only the state at its end, so the motion of $\pi_{0.5}$ never enters the trajectory ξ_k on which the reflection operates. The $\pi_{0.5}$ trajectory design in Table 5 places this motion into ξ_k.

Recorded states. $\pi_{0.5}$ predicts absolute joint and gripper targets and replans every 30 physics steps. At the end of each replanning interval, the harness reads the proprioception of the robot and records the end-effector pose of each arm, as a position and a unit quaternion in the planning frame of that arm, together with the gripper openings, the base pose, and the physics step. These fields and frames match the measured state that every end-effector command returns, so the MLLM can compare a state reached by $\pi_{0.5}$ with its own waypoints and command that state directly. The last entry of each policy call records the state at the end of that call, and because a physical robot measures these quantities at every control step, the record satisfies the information protocol of Section 3.1.

Exposure to the MLLM. The record reaches the MLLM through three channels. First, the result of each policy call lists one entry per replanning interval next to the existing step count, state, and images. Second, the memory tools that list earlier calls and measured robot states return the same entries from the episode record, so the MLLM can query them during the episode and during reflection. Third, the reflection prompt after the episode includes the entries of every policy call, so the record remains available for reflection in long episodes. All other settings, including the tool descriptions, the size budget of the working memory, the step limits, and the $\pi_{0.5}$ checkpoint, match the text design.

Results. With the executed trajectory, the harness succeeds in 17.5% of the episodes, down from 25.0%, while the use of the harness stays nearly unchanged. Every policy call of this design returns its executed trajectory, yet $\pi_{0.5}$ executes 41% of the commanded physics steps, against 42% with the text design, the final working memories store poses about as often, and the MLLM commands positions near the states that $\pi_{0.5}$ reached about as often as with the text design. These results indicate that visibility of the motion of $\pi_{0.5}$ alone changes neither how the MLLM divides control nor what its working memory records.

E.4 Comparison with the Critic-Driven RSI Loop

Protocol. The critic-driven RSI loop starts from the harness with $\pi_{0.5}$ and an empty playbook. In each of five rounds, ten training tasks run once with and once without their demonstration, 100 episodes in all, and a read-only critic instance of the same model inspects these episodes and accepts revisions to the playbook and the harness. Its starting point (Start) and its final version (Final) are evaluated on the first five test scenes of each of 18 tasks. The fixed working memory of the train–test protocol is scored on the same 90 scenes.

Results. As Table E.2 shows, the fixed working memory reaches a mean score of 92.1, against 64.4 for Final and 61.3 for Start, and matches or exceeds Final on 17 of the 18 tasks. The largest differences arise on shop checkout and utensil placement, where Final fails in all five episodes and the fixed working memory succeeds in four.

Table E.2: Successes out of 5 on the 90 test episodes shared with the critic-driven RSI loop, for the loop with an empty playbook (Start) and after five rounds (Final), and for the fixed working memory of SimplEvo under the train–test protocol. Bold marks the best value in each row.

Task	Start	Final	SimplEvo
apple to fruit bowl	**5**	**5**	**5**
bottle	0	0	**3**
bowl to plate	**5**	4	**5**
collect coffee beans	0	2	**5**
dishwasher	2	**4**	**4**
flip cup collect cookies	**2**	**2**	**2**
install gear	1	2	**4**
make sandwich	4	**5**	3
microwave	2	3	**4**
peg in hole	0	0	**3**
pen	**1**	0	**1**
perfume to cosmetics rack	**5**	4	**5**
put glass in glassbox	3	**4**	**4**
shop	0	0	**4**
soap to dish	**5**	**5**	**5**
teacup to saucer teapot to tray	3	2	**5**
tighten nut	1	0	**3**
utensils to holder	1	0	**4**
Successes of 90	40	42	**69**
SR (%)	44.4	46.7	**76.7**
Mean score	61.3	64.4	**92.1**

E.5 Matched Controls per Task

Table E.3: Successes out of 5 on the last five test episodes of each multi-layout EBench task with task memory only and with the fixed working memory of SimplEvo, paired by scene. Bold marks the better value in each row.

Task	Task memory only	SimplEvo
apple from shelf	**5**	**5**
apple to fruit bowl	**5**	**5**
bookmark on book	**5**	4
bottle	0	**2**
bowl to plate	4	**5**
detergent	4	**5**
dish	4	**5**
dishwasher	0	**5**
fruit	2	**5**
make sandwich	2	**3**
microwave	**4**	**4**
pen	**1**	0
perfume to cosmetics rack	**5**	**5**
remote to holder	**5**	**5**
salt to spice rack	**5**	**5**
shop	2	**4**
soap to dish	**5**	**5**
teacup to saucer teapot to tray	**5**	**5**
utensils to holder	1	**4**
Successes of 95	64	**81**
SR (%)	67.4	**85.3**

Table E.4: Successes out of 3 on the first three RoboDojo test layouts of each task in native order (126 episodes), with task memory only (TM) and with the fixed working memory of SimplEvo (WM), paired by layout. Gen., Prec., Long, Mem., and Open denote the five capability dimensions.

Task	Dim.	TM	WM	Task	Dim.	TM	WM	Task	Dim.	TM	WM
arrange largest number	Gen.	0	3	build tower	Prec.	2	2	cover blocks	Mem.	3	3
fold clothes	Gen.	2	3	deposit coin	Prec.	2	2	imitate sorting sequence	Mem.	1	2
hang mugs	Gen.	0	0	fasten screws	Prec.	1	0	match and pick from conveyor	Mem.	3	3
make toast	Gen.	0	1	insert key	Prec.	0	0	press by number	Mem.	3	3
pack objects into box	Gen.	0	0	insert tubes	Prec.	1	1	swap T	Mem.	0	3
pour liquid into cup	Gen.	3	2	play Xylophone	Prec.	1	3	swap blocks	Mem.	3	3
push T	Gen.	1	0	plug in charger	Prec.	0	1	align blocks	Open	2	3
sort nesting dolls by size	Gen.	0	3	pour balls into vase	Prec.	0	2	classify objects by language	Open	2	2
stack blocks	Gen.	3	3	classify objects	Long	2	3	general pickup	Open	3	2
stack bowls	Gen.	1	3	fill egg holder	Long	0	1	pick from conveyor by image	Open	0	2
store laptop and headphones	Gen.	0	0	fill pen holder	Long	0	0	pour by language	Open	0	3
sweep blocks	Gen.	0	1	make kong	Long	0	3	solve equation	Open	2	2
				organize table	Long	0	1	stack blocks by language	Open	1	3
				play stacking toy	Long	0	2	store tools in toolbox	Open	0	0
				play tic tac toe	Long	3	3				
				put bottles into dustbin	Long	3	3				

SR over the five dimensions: 40.83% with task memory only and 66.11% with the fixed working memory.

EBench. The matched control runs GPT-6-Astra with task memory only on the last five test episodes of each of the 19 multi-layout tasks and pairs each episode with the episode of the fixed working memory on the same scene, with the results in Table E.3. The fixed working memory raises the mean score from 84.5 to 94.7. The largest gains occur on dishwasher loading (from 0 to 5), the fruit task (from 2 to 5), and utensil placement (from 1 to 4). It also lowers the mean tokens per episode from 3.82M to 2.32M.

RoboDojo. This control uses the first three test layouts of each task in native order and pairs task memory only with the fixed working memory on each layout, as Table E.4 reports. The fixed working memory reaches a higher SR on 20 tasks and a lower one on 4, and 18 tasks tie. Five tasks rise from 0 to 3 of 3: arranging the largest number, sorting nesting dolls, making a kong, swapping a T-shaped block, and pouring by language.

E.6 Coordinate Replay and Its Threshold

Table E.5: Median share of moving waypoints per EBench test episode that lie within 1, 2, or 3 cm of a coordinate in the fixed working memory. Values in parentheses give the baseline floor obtained by scoring the corresponding direct-control episode (which reads no working memory) against the same stored coordinates. Rows report successful episodes on single-layout tasks, successful episodes on seen and unseen layouts of multi-layout tasks, and all 510 test episodes.

Episodes	n	1 cm	2 cm	3 cm
Single layout, succ.	109	0.76 (0.03)	0.82 (0.15)	0.88 (0.34)
Seen layout, succ.	93	0.22 (0.00)	0.45 (0.07)	0.66 (0.16)
Unseen layout, succ.	215	0.11 (0.00)	0.28 (0.00)	0.45 (0.05)
All episodes	510	0.25 (0.00)	0.45 (0.07)	0.63 (0.18)

Table E.5 counts as moving waypoints the arm targets that shift by more than 1 cm from the preceding target of the same arm. On single-layout tasks, successful episodes replay a median of 76% of their moving waypoints within 1 cm of a stored coordinate, far above the 3% baseline floor of direct control without working memory, and this gap holds at 2 and 3 cm. By contrast, on unseen layouts of multi-layout tasks, the median replay share at 1 cm drops to 11%.

E.7 Failure Modes of the Working Memory

Most failures with the fixed working memory follow two patterns, a stored recipe that reproduces failing executions as well as successful ones, and a lesson drawn from training successes whose cause the camera views leave ambiguous. Of the 93 failures among the 510 test episodes of the train–test protocol, 41 replay stored coordinates, with at least a quarter of their moving waypoints within 1 cm of a coordinate in the fixed working memory, and 27 of these 41 failures occur on

single-layout tasks. On nut tightening, each of the six failures places more than half of its moving waypoints near a stored coordinate, as the 14 successes do. Pen placement shows the second case. In four of its seven training successes, the official success arrived during a late adjustment of a pen that already appeared to stand in the holder, and the working memory states that "a late small intervention can precede success without proving its exact mechanism" while still recommending such adjustments. With this working memory, pen placement succeeds in 5 of 20 test episodes, and each of its 15 failures replays at most 5% of its moving waypoints. Pen placement and utensil placement together account for 23 of the 52 failures without replay.

F TRANSFER PROTOCOL

Table F.1: SR (%) and mean score of four MLLMs on two EBench tasks, 20 episodes each, with task memory only and with the working memory that GPT-6-Astra wrote at the same chain position.

	Task memory only		Task + working memory	
Model	**SR**	**Score**	**SR**	**Score**
Put glasses into a glassbox				
Gemini 3.8 Flash	0.0	15.0	**90.0**	**91.7**
MiMo-V2.6-Pro	0.0	6.7	70.0	78.3
DeepSeek-V4.1-Flash	0.0	6.7	40.0	45.0
GLM-5.3-Flash	0.0	3.3	20.0	25.0
Place utensils in a holder				
Gemini 3.8 Flash	20.0	40.0	**35.0**	**57.5**
MiMo-V2.6-Pro	0.0	0.0	10.0	12.5
DeepSeek-V4.1-Flash	0.0	10.0	5.0	20.0
GLM-5.3-Flash	0.0	5.0	5.0	10.0

Mixed generalization setting. For Table 7, each MLLM attempts the five test episodes in the Mixed generalization setting of every EBench task in two independent direct-control sessions. One session reads the task memory only, and the paired session reads the fixed working memory that GPT-6-Astra reads in Table 1. All MLLMs receive the same observation space, action tools, task memory, and episode prompt. The gains hold across task layouts. On the 35 single-layout episodes, the working memory lifts Gemini 3.8 Flash from 5 to 18 successes, DeepSeek-V4.1-Flash from 0 to 2, and GPT-6-Astra from 9 to 26. On the 95 multi-layout episodes, it lifts them from 26 to 52, from 1 to 15, and from 70 to 80, respectively.

Chain study with four MLLMs. To examine how transfer develops as the working memory accumulates experience from scratch, four recipient MLLMs attempt 20 scenes of putting glasses into a glassbox and of placing utensils in a holder twice in independent direct-control sessions. At chain position $k \in \{1, \ldots, 20\}$, one session reads the task memory M^{task} only, and the paired session additionally reads the working memory snapshot M_{k-1}^{work} that GPT-6-Astra wrote after episode $k-1$ along its 20-episode adaptation chain on that task, so both sessions of the first position read the task memory alone. As Table F.1 shows, the working memory improves all four recipients, with Gemini 3.8 Flash reaching 25 of 40 successes, up from 4, and MiMo-V2.6-Pro reaching 16, up from 0.

Replay by recipients. The replay share of a recipient episode is the fraction of its moving waypoints, defined as in Appendix E.6, that lie within 1 cm of a coordinate written in the working memory of its position. The median share on glasses packing is 0.61 for Gemini 3.8 Flash, 0.59 for MiMo-V2.6-Pro, 0.27 for GLM-5.3-Flash, and 0.17 for DeepSeek-V4.1-Flash, against at most 0.05 for the same models with task memory only, measured against the same coordinates. On utensil placement, no recipient episode reaches a share of 0.5, yet following the distilled lessons still raises the SR of four recipients.

Low-level gripper behavior. The gripper statistic of Section 5.4 pairs DeepSeek-V4.1-Flash with and without the working memory that GPT-6-Astra wrote after 15 episodes of its adaptation chain, on 24 episodes of eight tasks. Among the waypoints that raise the hand, it counts those with a gripper target between closed and fully open, a setting that holds an object at its measured width without squeezing it.

G Cost and Wall-Clock Time

Table G.1: Mean token composition per episode of GPT-6-Astra on EBench, with the cached share of input tokens, uncached input and output tokens per episode, and tokens per success, which divide all tokens of an evaluation, failed episodes included, by its successes. The training row covers the 192 episodes of the 26 adaptation chains, reflection turns included.

Configuration	Episodes	Successes	Tokens/ep. mean (M)	Tokens/ep. median (M)	Cached input (%)	Uncached input (k)	Output (k)	Tokens/ success (M)
Direct control	510	237	3.77	3.12	91.5	75.6	11.2	8.12
Harness with $\pi_{0.5}$	510	269	1.83	1.34	96.5	64.2	4.6	3.46
Harness with InternW0-Δ	510	284	2.22	1.76	97.0	65.7	4.9	3.99
SimplEvo, fixed working memory (test)	510	417	2.26	1.38	97.1	64.3	7.6	2.76
SimplEvo, adaptation chains (training)	192	101	4.09	3.47	97.1	118.1	14.4	7.77

Table G.2: Median wall-clock minutes per EBench test episode by task group, measured from the start to the end of agent control, excluding simulator queueing and scene resets. Bold marks the shortest median in each group.

Group	n	Direct	$\pi_{0.5}$	InternW0-Δ	SimplEvo
All	510	36.7	23.9	36.5	**14.3**
Tabletop	140	52.5	27.5	50.0	**15.1**
Mobile	370	29.0	21.3	32.0	**13.8**
Short horizon	220	20.6	12.7	20.5	**7.4**
Long horizon	290	53.5	33.3	63.2	**19.9**
High precision	80	48.2	24.7	35.4	**13.8**

Tokens on EBench. As Table G.1 shows, prompt caching dominates the token composition of every configuration. Because the agent resends its growing context at every turn of an episode, 91.5% to 97.1% of the input tokens repeat an earlier prefix and are served from the cache. The fixed working memory spends 2.26M tokens per test episode, more than the two harnesses with policy, in which one policy call advances many physics steps without an agent turn, and fewer than direct control.

Tokens on RoboDojo. Table G.3 gives the corresponding values on RoboDojo. The fixed working memory needs no reflection turn, which keeps its tokens per episode below those of online adaptation, 1.49M against 1.73M.

Table G.3: Tokens of GPT-6-Astra on RoboDojo per episode and per success, for task memory only and online adaptation on the 630 episodes of the 15 training layouts and for the fixed working memory on the 2,100 test episodes, with the cached share of input tokens and the share of reflection turns. For task memory only, the tokens cover the control phase of each episode. Tokens per success follow Table G.1.

Configuration	Eps.	Succ.	Tokens/ ep. (M)	Cached (%)	Reflection (%)	Tokens/ succ. (M)
Task memory only	630	216	1.41	95.3	–	4.12
Online adaptation	630	350	1.73	95.7	11.9	3.11
Fixed working memory (test)	2,100	1,401	1.49	95.7	–	2.23

Wall-clock time. On EBench, the fixed working memory has the shortest median episode duration in every task group, 14.3 minutes over all test episodes, compared with 23.9 minutes for the harness with $\pi_{0.5}$ and 36.7 minutes under direct control, as listed in Table G.2. On the 2,100 RoboDojo test

episodes, the median episode with the fixed working memory takes 7.0 minutes, most of it in model turns.

H CASE STUDIES

This section extends Figure 3 of Section 5.3 and the failure analysis of Appendix E.7 with further test episodes of SimplEvo with the fixed working memory from the train–test evaluations of Tables 1 and 2. Figure H.1 shows coordinate replay on single-layout tasks, Figure H.2 lesson-guided execution on unseen layouts, and Figure H.3 three failure modes. Every memory excerpt and agent message in the figures is copied verbatim from the episode record.

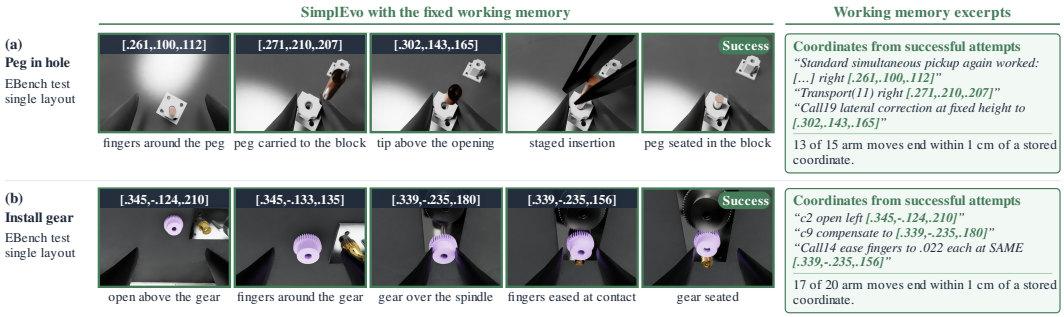

Figure H.1: Coordinate replay on two high-precision single-layout tasks of EBench, peg insertion and gear installation. Both episodes of SimplEvo with the fixed working memory (green) succeed, and direct control and the harness with $\pi_{0.5}$ fail on the same scenes. Dark labels indicate commanded positions in meters that match coordinates in the working memory, the cards quote the lines that store them, and [. . .] marks internal omissions. Following Appendix E.6, arm moves count the moving commanded waypoints of the episode, whose target lies more than 1 cm from the previous target of the same arm. Frames come from the wrist cameras, and the last frame shows the terminal state.

Coordinate replay. On single-layout tasks, the coordinates stored from successful training attempts carry a test episode from the grasp to the final insertion, as Figure H.1 shows. In the peg-insertion episode, 13 of the 15 arm moves end within 1 cm of a stored coordinate, covering the peg grasp, the transport to the held block, and the lateral correction above the hole. The gear-installation episode reissues the stored poses for opening the gripper above the gear, aligning the gear over the spindle, and easing the fingers at contact, with 17 of its 20 arm moves matching a stored coordinate within 1 cm.

Lessons on unseen layouts. When the test layout differs from every training scene, most commanded positions lie farther than 1 cm from every stored coordinate, and the episodes follow the lessons of the working memory, as Figure H.2 illustrates. On the unseen fruit-milkshake layout, none of the 24 arm moves lies within 1 cm of a stored pose. After the first tilt leaves the rim of the mug right of the cup opening, the agent shifts the mug left, slightly back, and lower, as the stored lesson on fore-aft alignment advises, and the episode succeeds. On the unseen teacup layout, the agent lifts the cup by 1 cm before transport and lowers the teapot while monitoring the pot body, following two stored lessons. On the unseen conveyor layout, an initial approach tips the dog, and the agent retracts and locates the dog again before a flat torso grasp, as a stored recovery lesson advises. On the unseen equation layout, the agent grips the digit tile along its length, following the stored grasp lesson, and carries it at the stored pad pose.

Failure modes. The three failures in Figure H.3 arise from stored coordinates that do not separate successful from stalled attempts, from a visual completion judgment that contradicts a stored lesson, and from a post-assembly state that needs a recovery absent from the stored attempts. On nut tightening, the working memory stores seating poses of successful and stalled training attempts a few millimeters apart and also notes that absolute XYZ recipes remain unreliable. The test episode places the nut repeatedly within a few millimeters of a stored coordinate, and no placement threads it before the step limit. On pen placement, the working memory states that overlap, tilt, or a gap

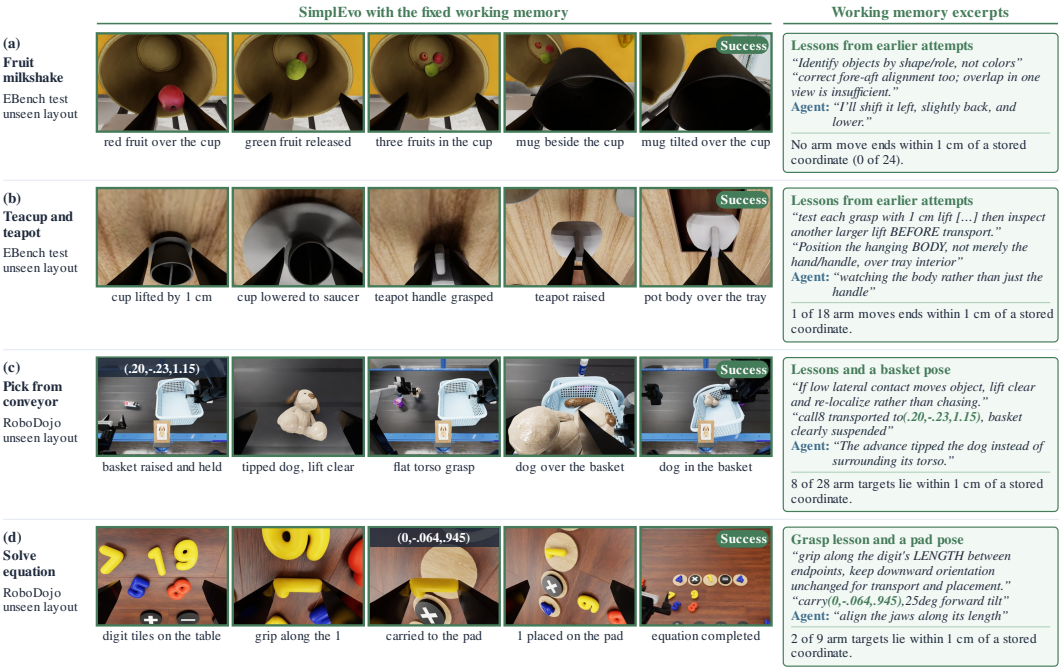

Figure H.2: Lessons on unseen layouts. (a, b) EBench test scenes of the fruit milkshake task and of the teacup and teapot task, whose layouts appear in no training episode of the chain. (c, d) RoboDojo test layouts of picking from a conveyor by image and of solving an equation. All four episodes of SimplEvo with the fixed working memory (green) succeed. On the same scenes, direct control and the harness with $\pi_{0.5}$ fail in (a) and (b), and GPT-6-Astra with task memory only fails in (c) and (d). The cards quote lessons of the working memory read in the episode, and *Agent* marks a message of the agent in the same episode. Dark labels give commanded positions in meters that equal stored coordinates. EBench counts cover the moving commanded waypoints, and RoboDojo counts cover the arm targets away from the home pose. Frames come from the wrist cameras, and in (c) and (d) also from the head camera.

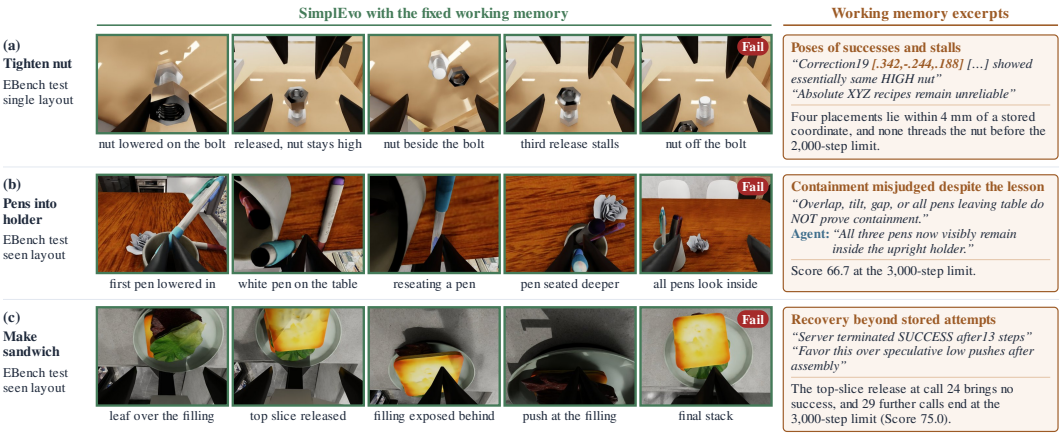

Figure H.3: Failures of SimplEvo with the fixed working memory on three EBench tasks: nut tightening (score 50.0), pens into a holder (score 66.7), and sandwich making (score 75.0). Each episode ends at the physics step limit of its task. The cards quote the working memory read in the episode, and *Agent* marks the last message of the agent in the episode. Frames come from the wrist cameras.

do not prove containment, yet the last message of the agent reports all three pens inside the holder, and the episode ends at the step limit with a score of 66.7. On sandwich making, the three latest successful training attempts in the working memory end when the top slice is released. In the test episode, the release of the top slice does not complete the task, and the agent spends the rest of the episode on inspection and corrective pushes, against the stored advice to avoid speculative low pushes after assembly.